Check for updates

Article

The International Journal of Robotics Research
2024, Vol. 43(13) 2049–2069
© The Author(s) 2024

Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/02783649241246557
journals.sagepub.com/home/ijr

Reactive collision-free motion generation in joint space via dynamical systems and sampling-based MPC

Mikhail Koptev[1], Nadia Figueroa[2] and Aude Billard[1]

Abstract

Dynamical system (DS) based motion planning offers collision-free motion, with closed-loop reactivity thanks to their analytical expression. It ensures that obstacles are not penetrated by reshaping a nominal DS through matrix modulation, which is constructed using continuously differentiable obstacle representations. However, state-of-the-art approaches may suffer from local minima induced by non-convex obstacles, thus failing to scale to complex, high-dimensional joint spaces. On the other hand, sampling-based Model Predictive Control (MPC) techniques provide feasible collision-free paths in joint-space, yet are limited to quasi-reactive scenarios due to computational complexity that grows cubically with space dimensionality and horizon length. To control the robot in the cluttered environment with moving obstacles, and to generate feasible and highly reactive collision-free motion in robots' joint space, we present an approach for modulating joint-space DS using sampling-based MPC. Specifically, a nominal DS representing an unconstrained desired joint space motion to a target is locally deflected with obstacle-tangential velocity components navigating the robot around obstacles and avoiding local minima. Such tangential velocity components are constructed from receding horizon collision-free paths generated asynchronously by the sampling-based MPC. Notably, the MPC is not required to run constantly, but only activated when the local minima is detected. The approach is validated in simulation and real-world experiments on a 7-DoF robot demonstrating the capability of avoiding concave obstacles, while maintaining local attractor stability in both quasi-static and highly dynamic cluttered environments.

Keywords

Motion and path planning, collision avoidance, reactive and sensor-based planning

Received 8 June 2023; Revised 12 February 2024; Accepted 16 March 2024
Associate Editor: Takayuki Osa

1. Introduction

Modern robotic systems are expected to operate in a variety of environments and interact with static and dynamic objects, as well as other agents, such as humans, animals, and other robots. While direct contact may be necessary to accomplish certain tasks, almost any motion requires a reaching phase where the robot must move its end-effector to a desired location while avoiding collisions between the robot body and obstacles or and other agents. For future reference, we assume that the obstacles in the environment are moving unpredictably, and reactive motion assumes feasible plan generation on frequencies above 100 Hz.

Techniques to generate collision-free motion can be categorized into two paradigms: offline and feedback/reactive planning (LaValle, 2006). In the former, the problem is divided into two phases—offline planning of collision-free paths (typically sampling-based algorithms or trajectory optimization) followed by trajectory execution (Spong et al., 2020). However, depending on the complexity of the environment and the dimensionality of the robot configuration space, offline planning may take a significant amount of time, which hinders operation in dynamic environments with moving obstacles and active agents.

Early works in reactive collision-free motion generation rely on local alternation of the system's dynamics in the vicinity of obstacles via Artificial Potential Fields (Khatib, 1986) or navigation functions (Rimon and Koditschek, 1992). Such classical feedback motion planning techniques, although reactive, are vulnerable to local minima and limited to parametric obstacle representations such as convex

[1]Learning Algorithms and Systems Laboratory (LASA), École Polytechnique Fédérale de Lausanne (EPFL), Lausanne, Switzerland
[2]Department of Aeronautics and Astronautics, University of Pennsylvania, Philadelphia, PA, USA

Corresponding author:
Mikhail Koptev, Learning Algorithms and Systems Laboratory (LASA), École Polytechnique Fédérale de Lausanne (EPFL), EPFL-STI-IMT-LASA, Station 9 CH, Lausanne 1015, Switzerland.
Email: mikhail.koptev@epfl.ch

(spheres, ellipsoids, etc) or star-shaped obstacles. To ensure reactive collision-free motions with convergence guarantees the modulated dynamical system (DS) based approach was introduced by Khansari-Zadeh and Billard (2012). In this approach, the nominal robot motion is generated by a DS that defines a vector field in the task space for end-effector motion (in this case, an additional Inverse Kinematics controller is required) or in joint space to describe the whole-body state-dependent motion law. The proposed modulation locally changes this law in order to navigate around obstacles.

Local modulation of DS for obstacle avoidance is performed with relatively low computation cost, however, convergence guarantees can only be ensured for conservative obstacle types such as convex (Khansari-Zadeh and Billard, 2012) or star-shaped (Billard et al., 2022; Huber et al., 2019). Further, a saddle point local minimum trajectory still remains for such parametric shapes. These shortcomings limit the applicability of reactive DS modulation approaches to complex spaces with arbitrarily shaped obstacles that are hard to convexify such as joint space of redundant manipulators.

Receding horizon path planning approaches such as Model Predictive Control (MPC), can be used to generate collision-free trajectories in real-time (Erez et al., 2013; White et al., 2022). However, these methods are often computationally expensive, and finding a balance between computation speed and trajectory quality can be challenging (Bhardwaj et al., 2022). The computational complexity of MPC methods grows cubically with state dimensionality and time horizon (Richter et al., 2012), which can be prohibitive for cases when the robot with a high number of degrees of freedom (DoF) is required to reactively navigate in cluttered environments while avoiding obstacles.

1.1. Contributions

In this paper, we aim to combine the advantages of DS motion planning and sampling-based MPC to allow for instantaneous obstacle avoidance and swift feasible trajectory generation for any type of environment. We adopt a modulated DS approach (Billard et al., 2022; Khansari-Zadeh and Billard, 2012) as a primary motion generator and empower it with a popular sampling-based MPC algorithm known as Model Predictive Path Integral (MPPI) control (Williams et al., 2017), to navigate the robot away from the local minima caused by concave obstacles. We prove theoretically that the impenetrability and local convergence properties of the modulated DS are preserved with the MPC-based additive terms. Additionally, we focus on the challenging problem of joint space control and validate the proposed method using a 7-DoF robot. Example of a reaching task executed by the robot is shown in Figure 1. Apart from presenting a simulated benchmark, we demonstrate the collision avoidance controller running at 500 Hz (on a modern laptop CPU) with a real robot, generating reaching motions while avoiding the human that tries to obstruct the robot reaching motion with a concave arm trap. The source code for running the presented algorithm in simulation and on real robot is available online.[1]

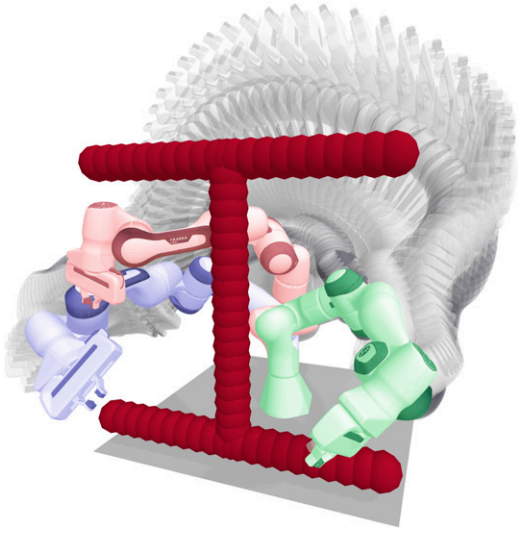


Figure 1. The robot begins in the blue state (start joint configuration) and must reach the attractor configuration (green state) while avoiding a concave task-space obstacle represented by a set of red spheres. The nominal robot dynamics is a linear motion in joint space towards the attractor. The standard modulated DS approach (Khansari-Zadeh and Billard, 2012) exhibits a local minimum (represented by the red robot state) in this scenario. Our proposed method is able to reactively navigate around the obstacle and reach the attractor successfully.

1.2. Paper organization

In Section 2, we summarize related works for task space and joint space robot motion planning in the presence of obstacles. Section 3 presents the mathematical problem formulation, including assumptions, definitions, and goals of this paper. In Section 4, we present the background of the DS modulation method, as it is the basis of our approach. Our method formulation is then presented in Section 5. Further, Section 6 introduces the basics of the state-of-the-art MPPI algorithm, and describes our approach to leveraging optimization to find optimal DS modulation strategies. Section 7 outlines various implementation details, and presents our approach to implementing the hybrid controller combining modulated DS and MPPI methods. Finally, Section 8 presents the validation of the method in simulation and on a real robot.

2. Related work

In this paper, we consider the generation of feasible paths in the configuration space of a robotic manipulator. For manipulators with revolute joints, the configuration space coincides with the robot's joint space. In this section we cover the state-of-the-art in motion planning in high-dimensional spaces, as well as collision detection techniques.

2.1. Offline path planning

A hierarchical approach, in which a feasible path is first generated offline and then executed by a high-frequency controller, can be considered the dominant paradigm in robotic systems. Several sampling-based methods exist to generate paths in the high-dimensional joint space of the robot, such as Rapidly-exploring Random Trees, Probabilistic Roadmaps, and their variants (Kavraki et al., 1996; Kuffner and LaValle, 2000; Sandakalum and Ang Jr., 2022). These methods provide probabilistic completeness, but their computation and memory costs are often significant (Kingston et al., 2018). The path generation time can range from seconds to minutes, depending on the complexity of the environment and the system's dimensionality (Chase et al., 2021). Although these methods are suitable for many applications, they may not be the best choice for rapidly changing environments that require online replanning at frequencies exceeding 100 Hz. Trajectory optimization methods, such as CHOMP (Ratliff et al., 2009) and TrajOpt (Schulman et al., 2014), can optimize for feasible trajectories within a tight, frequently sub-second time budget, providing control frequency of 1–5 Hz. However, quick optimization convergence requires a good initial guess for the trajectory (Zucker et al., 2013), which can be problematic in a changing environment. Due to the amount of time needed to calculate a trajectory, these methods are unsuitable to use in highly dynamic environments with moving obstacles, and are mainly applied in a static environments with pre-computed distance fields (Yang et al., 2019).

2.2. Receding horizon path planning

An approach that differs from hierarchical model involves repetitive solving of a nonlinear optimization problem with a receding horizon. It encapsulates tasks for the system and system dynamics as constraints in nonlinear optimization problem and generates dynamically feasible trajectories online. Such methods have wide range of applications, including humanoid robot control (Erez et al., 2013) and autonomous driving (Frasch et al., 2013). The main limitation of MPC methods lies in polynomial scaling with number of parameters in the optimization. Typically, the computational cost has complexity O (n^3), where n is number of parameters determined by number of states, control inputs, and time horizon (Domahidi et al., 2012; Richter et al., 2012). Thus, the operating frequency of these methods typically does not exceed 20 Hz for high-DoF robots. As this approach is unable to provide replanning at 100 Hz rate, it cannot fulfill the objectives of this paper. However, such methods may be applied in a hybrid controller scheme to generate high-level motion policies (Hansel et al., 2023).

Recent developments in computational hardware, such as GPU chips allowing for massive parallelization, have enabled a new generation of optimization methods that rely on sampling-based exploration for approximating the optimal input distribution (Williams et al., 2017). Such methods are capable of handling complex dynamics and non-differentiable cost functions. For instance, Bhardwaj et al. (2022) use MPPI approach and demonstrate online trajectory optimization via sampling-based model predictive control operating at 125 Hz on a 7 DoF robotic manipulator. This method can be considered *reactive*, as it is capable of quickly moving the robot away from collisions if an obstacle moves towards the manipulator. At the same time, thanks to trajectory planning with a look-ahead horizon, this method can navigate relatively complex environments. However, the method may still require some specific sampling heuristics, or increased number of samples to find paths in cluttered environments (Sacks and Boots, 2022; Yoon et al., 2022). We use MPPI in our method in combination with sampling in locally obstacle-tangent space to achieve improved path-planning.

2.3. Reactive planning

Similar to the artificial potential fields approach (Khatib, 1986) mentioned in the introduction, control-barrier functions (CBF) method can be used to reactive motion generation (Singletary et al., 2021). However, CBF requires solving a QP problem on each iteration, thus reducing scalability to higher dimensions, such as robot joint space.

Dynamical systems, which represent a pre-defined state-dependent law acquired by learning from demonstrations or manually defined by the user, are another classical method for robot motion generation (Gribovskaya et al., 2011; Khansari-Zadeh and Billard, 2011; Kronander et al., 2015). They can encapsulate complex behaviors and guide the robot to follow the desired trajectory without extensive real-time computations (Figueroa and Billard, 2018; Figueroa et al., 2020; Shavit et al., 2018a). While DS approaches generally do not consider obstacles, an extension was proposed to enable navigation around convex (Khansari-Zadeh and Billard, 2012) and star-shaped (Huber et al., 2019) obstacles. This method relies on obstacle-related information, such as distance function and normal direction, to modulate the DS close to the obstacles. While many objects can be approximated with convex and star shapes, such approximations may be too restrictive in some cases. We build upon this method as it allows for reactive any-frequency collision avoidance.

In recent years, Riemannian Motion Policies (RMPs) (Ratliff et al., 2018) have emerged as an alternative to modular reactive motion generation, formulated as second-order DS with associated Riemannian metrics in various task spaces. For each objective or constraint, for example, target reaching, orientation control, obstacle avoidance, redundancy resolution, and joint limits, an individual RMP must be designed. To combine them into a global reactive motion policy while ensuring geometric consistency Cheng et al. (2018) introduced RMPflow, a computational graph representing tree-structured task-maps of RMPs. The global

motion policy synthesized by RMPflow is Lyapunov-stable if the underlying policies (subtasks) are a class of stable RMPs known as Geometric Dynamical Systems (GDS), the closure property relating to task consistency is preserved and task priorities are properly tuned. Such restrictions may lead to suboptimal performance and unnecessary energy consumption by the system. To alleviate this, Li et al. (2019) offer a relaxation of the GDS assumption and reinterpretation of the stability guarantees of RMPflows through the lens of control lyapunov functions. Van Wyk et al. (2022) further improved RMPflow by introducing geometric fabrics, which utilize Finsler geometries and bending terms to enhance expressivity of subtasks while ensuring stability and convergence to a local solution and transform trees to combine them. In a similar vein, Composable Energy Policies (CEP) (Urain et al., 2021) offer modular reactive motion generation by optimizing over products of stochastic policies, effectively resolving conflicts between multiple motion objectives via a Bayesian network energy tree.

While the latter methods offer desirable performance and stability guarantees, they require the construction of graphs or trees or entail multi-objective optimization, which can be computationally intensive compared to our modulation approach—which is a closed form solution that only requires local reshaping of a nominal DS. Furthermore, when performing collision avoidance of non-convex boundaries all of these methods are prone to local minima and must resort to adhoc environment heuristics (as in Van Wyk et al. (2022)) or learning the adhoc obstacle avoidance subtask behavior (as in Urain et al. (2021)). In contrast, the modulated DS framework offers a general approach for obstacle avoidance, avoiding the need for complex constructions, intensive optimizations or adhoc heuristics. We achieve couple the modulated DS approach with MPPI control, to achieve a practical and intuitive, yet powerful solution for reactive motion planning.

Our proposed approach shares similarities with the hybrid planning architectures introduced by Löw et al. (2021) and Hansel et al. (2023) that combine local reactive motion policies with longer horizon planning schemes. The former approach, named PROMPT incorporates probabilistic movement primitives (ProMPs) (Paraschos et al., 2013) in both sampling and optimization based trajectory planners to generate high quality feasible paths. This is achieved by leveraging the constraint conditioning and blending capabilities of ProMPs. Nevertheless, this approach is not applicable to reactive scenarios with highly dynamic obstacles nor is it scalable to high-dimensional spaces as its computational complexity increases relative to state space dimensionality, planning horizon and number of samples. This is evidenced by experiments reporting 1 Hz of planner runtime for 2D state spaces. The hierarchical policy blending as inference (HiPBI) approach (Hansel et al., 2023), built upon the RMPflow framework, addresses local minima through an online sampling-based planner. It reformulates RMPs as Gaussian policies and blends them using the product of experts technique. To dynamically optimize weights, a high-level, sampling-based online planner operates on the parameter space, mitigating local minima seen in the standard RMPflow approach. While our approach shares similarities with HiPBI, latter is constrained by RMPs, requiring specific subtask policy construction and a prior distribution assumption. Although HiPBI simplifies task-tree construction and weight tuning, it remains computationally intensive, reporting a 10 Hz planning frequency for a 7DoF manipulator robot. In contrast, our approach uses modulated DS for reactive motion policies, ensuring stability guarantees and achieving a 500 Hz execution rate. This choice enhances collision avoidance and reactivity, resulting in natural trajectories. The coupling with the high-level MPPI controller further enhances local minima free navigation in complex scenarios.

2.4. Differentiable collision checking

Although all the methods mentioned above can consider various constraints for generating paths, collision avoidance is usually the most critical constraint in motion planning. Importantly, the collision checking routine (and corresponding gradient computation, where needed) often takes up the majority of computation time (Pan and Manocha, 2016). Based on the implementation, speedups can be achieved by simplifying collision shapes (by means of convex hulls, geometric primitives, or spheres) (Stasse et al., 2008) or by pre-computing the static environment (Zucker et al., 2013). Recently, there were developments in approximating the distance functions using Machine Learning (ML) based methods (Koptev et al., 2021; Liu et al., 2022; Salehian et al., 2018; Zhi et al., 2022) for efficient use in robot control. In this paper, we rely on our previous work (Koptev et al., 2023), which learns a signed distance as a function of robot joint states and 3D points in the robot's environment. Properties such as batched evaluation or continuous differentiability enable efficient use of this function in the proposed method.

3. Problem definition

3.1. Assumptions and definitions

In this paper, we consider a robot with K physical links and d degrees of freedom (DoF). The joint state of the robot is defined as a vector of joint angles $\boldsymbol{q} = [q^1, \ldots, q^d] \in \mathbb{R}^d$. Additionally, it is assumed that all the joints of the robot are revolute and are subject to joint limits, $\boldsymbol{q} \in [\boldsymbol{q}_{min}, \boldsymbol{q}_{max}]$. The desired robot motion is defined by a continuous time-invariant nominal DS of the following form:

$$\dot{\boldsymbol{q}} = \boldsymbol{f}(\boldsymbol{q}), \tag{1}$$

which defines a state-dependent vector field guiding the robot state towards a goal (attractor $\boldsymbol{q}^* \in \mathbb{R}^d$). The function $\boldsymbol{f}(\boldsymbol{q})$ is autonomous, that is, the law governing the evolution of the system depends solely on the system's current state $\boldsymbol{q}$

and not on time. While the system defined in (1) is time-invariant, the system state $\boldsymbol{q} = \boldsymbol{q}(t)$ changes over time. For better readability, time-dependency of the state variable is omitted in the remainder of the text.

We assume that the nominal DS $\boldsymbol{f}(\boldsymbol{q})$ is known and globally asymptotically stable (G.A.S.) to the attractor $\boldsymbol{q}^*$ wrt. to a Lyapunov function $V(\boldsymbol{q}, \boldsymbol{q}^*) : \mathbb{R}^d \rightarrow \mathbb{R}$. Such G.A.S. DS can be designed by users (Salehian et al., 2018), or learned from a demonstration (LfD) (Figueroa and Billard, 2018; Khansari-Zadeh and Billard, 2011; Kronander et al., 2015; Shavit et al., 2018b), and can be of arbitrary form (e.g., linear, nonlinear). While the nominal DS may implicitly avoid collisions in some scenarios, such as when an LfD-generated trajectory avoids static obstacles, it is not always guaranteed that it will do so in all scenarios. As shown by Wang et al. (2022), DS-based LfD can ensure reachability to a target $\boldsymbol{q}^*$, but cannot ensure invariance to unsafe boundaries, unless a boundary function is explicitly defined and enforced through modulation or control-barrier functions (Taylor et al., 2020).

Obstacles (static or dynamic) can be defined as a set of points $\mathcal{O} \subset \mathbb{R}^3$ in the workspace of the robot. We assume that obstacles are known and are represented either by a point cloud or by a set of spheres. Moreover, we consider obstacles of arbitrary shape, including concave ones. Based on previous work (Koptev et al., 2023), we define the vector-valued function $\boldsymbol{\Gamma}(\boldsymbol{q}, \boldsymbol{y}) : \mathbb{R}^{d+n} \rightarrow \mathbb{R}^K$, which represents minimal distances between an n-dimensional point $\boldsymbol{y} \in \mathcal{O}$ and each link ($k = 1, \ldots, K$) of the robot in configuration $\boldsymbol{q}$; that is, $\boldsymbol{\Gamma}(\boldsymbol{q}, \boldsymbol{y}) = [\Gamma_1, \ldots, \Gamma_K]$. Function $\boldsymbol{\Gamma}(\boldsymbol{q}, \boldsymbol{y})$ is differentiable, and its partial derivative with respect to $\boldsymbol{q}$ results in the following Jacobian matrix:

$$\mathrm{Jac}_q(\boldsymbol{\Gamma}(\boldsymbol{q},\boldsymbol{y})) = \begin{bmatrix} \frac{\partial \Gamma_1(\boldsymbol{q},\boldsymbol{y})}{\partial q_1} & \cdots & \frac{\partial \Gamma_K(\boldsymbol{q},\boldsymbol{y})}{\partial q_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial \Gamma_1(\boldsymbol{q},\boldsymbol{y})}{\partial q_d} & \cdots & \frac{\partial \Gamma_K(\boldsymbol{q},\boldsymbol{y})}{\partial q_d} \end{bmatrix} \tag{2}$$

where each k-th column vector of $\mathrm{Jac}_q(\boldsymbol{\Gamma}(\boldsymbol{q}, \boldsymbol{y}))$ corresponds to the gradient of each k-th scalar distance function $\Gamma_k(\boldsymbol{q}, \boldsymbol{y})$ with respect to $\boldsymbol{q}$. For simplified notation, we use

$$\Gamma(\boldsymbol{q}) = \min_{\boldsymbol{y} \in \mathcal{O}} \min_{k=1,\ldots,K} \Gamma_k(\boldsymbol{q}, \boldsymbol{y}), \tag{3}$$

and denote the corresponding column of $\mathrm{Jac}_q(\boldsymbol{\Gamma}(\boldsymbol{q}, \boldsymbol{y}))$ as $\nabla\Gamma(\boldsymbol{q})$. In this way, $\Gamma(\boldsymbol{q}) : \mathbb{R}^{d+n} \rightarrow \mathbb{R}$ represents the minimal distance between the robot in configuration $\boldsymbol{q}$ and the nearest obstacle, and $\nabla\Gamma(\boldsymbol{q})$ represents the corresponding repulsive gradient in the joint space of the robot. Similarly, we assume the existence of function $\Gamma_{self}(\boldsymbol{q})$, which represents the minimal distance between the robot in configuration $\boldsymbol{q}$ and its own links. The gradient of $\Gamma_{self}(\boldsymbol{q})$ with respect to $\boldsymbol{q}$ is denoted as $\nabla\Gamma_{self}(\boldsymbol{q})$. Please refer to our previous work (Koptev et al., 2021, 2023) for more details on learning and applications of such distance functions.

3.2. Goals

The main goal of this paper is to design a modulation algorithm that reshapes $\boldsymbol{f}(\boldsymbol{q})$ from (1) with respect to the task-space obstacles $\mathcal{O}$ defined by $\Gamma(\boldsymbol{q})$ and self-collision boundary defined by $\Gamma_{self}(\boldsymbol{q})$, enabling the robot to (a) *avoid concave obstacles in joint space* while (b) *maintaining local stability of the nominal DS* (Equation (1)), and a controller capable of (c) *computing the required modulation with high frequency* to enable avoidance of moving obstacles.

To this end, we aim to leverage the parallelization and differentiability properties of the learned function $\Gamma(\boldsymbol{q})$, by combining the modulation framework (Huber et al., 2019; Khansari-Zadeh and Billard, 2012) with sampling-based model predictive control (Bhardwaj et al., 2022; Williams et al., 2017). The vision is to achieve the desired properties such as local stability of the modulated DS and the ability to reactively avoid moving obstacles of arbitrary shape, within a single unified framework.

4. Background

In this section, we provide the mathematical preliminaries describing the DS modulation method for obstacle avoidance, as it is essential for understanding our approach. We begin in Section 4.1 by introducing the original DS modulation method by Khansari-Zadeh and Billard (2012) that ensures impenetrability of obstacle boundaries, yet suffers from spurious attractor generation. Next, in Section 4.2 we describe the improved approach by Huber et al. (2019) designed to ensure convergence for convex and star-shaped obstacles (a unique subset of non-convex obstacles) almost everywhere except at a single saddle point trajectory. We further interpret this improved method from a controlled system perspective, setting the groundwork for our novel *locally deflected obstacle-tangent space dynamics* approach, that allows circumnavigation of complex non-convex obstacles in high-dimensional state spaces, detailed in Section 5. For an in-depth description of DS as motion policies and their robotic applications, please refer to (Billard et al., 2022).

4.1. Dynamical system modulation

A nominal Dynamical System, as in equation (1), defines a state-dependent vector field that can be altered, or modulated, by rotating the field with a modulation matrix $\boldsymbol{M}(.)$, where $\boldsymbol{M}$ can depend on different variables. The *modulated Dynamical System* is then defined as

$$\dot{\boldsymbol{q}} = \boldsymbol{M}(\,\cdot\,)\boldsymbol{f}(\boldsymbol{q}), \tag{4}$$

Depending on the choice of $\boldsymbol{M}(.)$, this DS can exhibit various behaviors. For example, $\boldsymbol{M}(\boldsymbol{q})$ is a matrix-valued function of the system state $\boldsymbol{q}$, and it can be activated differently in different regions of the state space, reshaping the nominal DS $\boldsymbol{f}(\boldsymbol{q})$. Importantly, if we aim to preserve

some stability properties of nominal DS, certain limitations must be applied to the modulation matrix $\boldsymbol{M}(.)$.

When starting from a G.A.S. DS $\boldsymbol{f}(\boldsymbol{q})$, reshaping it with a full rank and locally active matrix $\boldsymbol{M}(\boldsymbol{q})$ only guarantees that the system remains stable (no spurious attractors are introduced but limit cycles can arise) and that trajectories can be kept arbitrarily close to the attractor $\boldsymbol{q}^*$, if they start close enough (which is equal to local asymptotic stability of the attractor) (Kronander et al., 2015).

DS modulation using matrix $\boldsymbol{M}(\boldsymbol{q}, \mathcal{O}) \in \mathbb{R}^{d\times d}$ that depends on the obstacles configuration $\mathcal{O}$, can be used to achieve obstacle avoidance. Further, we omit the matrix $\boldsymbol{M}(\boldsymbol{q})$ dependency on obstacles state $\mathcal{O}$ for a simpler notation, and always assume that $\boldsymbol{M}$ is a function of both state $\boldsymbol{q}$ and obstacle configuration $\mathcal{O}$. To allow for obstacle avoidance, the following modulation matrix is proposed by Khansari-Zadeh and Billard (2012):

$$\boldsymbol{M}(\boldsymbol{q}) = \boldsymbol{E}(\boldsymbol{q})\boldsymbol{D}(\boldsymbol{q})\boldsymbol{E}(\boldsymbol{q})^{-1}, \tag{5}$$

where $\boldsymbol{M}(\boldsymbol{q})$ is composed through eigenvalue decomposition. In (5), $\boldsymbol{D}(\boldsymbol{q})$ is a diagonal scaling matrix and $\boldsymbol{E}(\boldsymbol{q})$ is an orthogonal matrix defined as

$$\boldsymbol{E}(\boldsymbol{q}) = [\,\boldsymbol{n}(\boldsymbol{q}) \quad \boldsymbol{e}_1(\boldsymbol{q}) \quad \dots \quad \boldsymbol{e}_{d-1}(\boldsymbol{q})\,]. \tag{6}$$

The first column of the matrix $\boldsymbol{E}(\boldsymbol{q})$ consists of the obstacle normal $\boldsymbol{n}(\boldsymbol{q})$.[2] The remaining columns, $\boldsymbol{e}_i(\boldsymbol{q})$, form a $(d-1)$-dimensional orthonormal basis for the tangent space at the state $\boldsymbol{q}$. The tangential hyperplane formed by this basis is referred to as the *deflection* hyperplane by Khansari-Zadeh and Billard (2012). Note that it is not a necessary condition for this basis to be orthonormal. Any basis that forms a tangential hyperplane to the normal $\boldsymbol{n}(\boldsymbol{q})$ and has linearly independent columns is valid. However, by enforcing orthonormality on the basis, we can exploit the fact that $\boldsymbol{E}(\boldsymbol{q})$ becomes orthogonal and $\boldsymbol{E}(\boldsymbol{q})^{-1} = \boldsymbol{E}(\boldsymbol{q})^T$, thereby improving numerical stability.

The diagonal matrix $\boldsymbol{D}(\boldsymbol{q})$ is defined as

$$\boldsymbol{D}(\boldsymbol{q}) = \begin{bmatrix} \lambda_n(\boldsymbol{q}) & 0 & \dots & 0 \\ 0 & \lambda_\tau(\boldsymbol{q}) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_\tau(\boldsymbol{q}) \end{bmatrix}, \tag{7}$$

and is composed of two eigenvalues $\lambda_n(\boldsymbol{q})$ and $\lambda_\tau(\boldsymbol{q})$ defined as a function of the distance of the system from the obstacle $\Gamma(\boldsymbol{q})$.

While the definition of $\Gamma(\boldsymbol{q})$ may vary, the general idea is that $\boldsymbol{D}(\boldsymbol{q})$ should become identity matrix when the system is far from collisions, $\lambda_n(\boldsymbol{q})$ should approach 0 as the system is getting closer to collision, and $\lambda_n(\boldsymbol{q}) = 0$ at the obstacle boundary $\boldsymbol{Q}_\mathcal{O}$. In essence, the modulation (5) reshapes the flow of the nominal DS (1) based on proximity to the obstacle, thus allowing obstacle avoidance. Refer to Figures 2 and 3 for 2-dimensional visualization of DS modulation.

However, the collision avoidance properties of such modulation are restricted to a subset of convex obstacles. There is also an edge case within the subset of states located on the obstacle boundary $\boldsymbol{Q}_\mathcal{O}$. To ensure *obstacle impenetrability* by design, $\lambda_n(\boldsymbol{q}) = 0$ for all $\boldsymbol{q} \in \boldsymbol{Q}_\mathcal{O}$, causing the matrix $\boldsymbol{M}(\boldsymbol{q})$ to lose rank and create a nullspace. Consequently, velocities may vanish, leading to the emergence of spurious attractors and destabilization of the system's dynamics. Figure 3 demonstrates how vanishing tangential component may lead to a local minima, rendering DS modulation framework unable to ensure convergence.

Definition 1. (Neumann boundary condition). *Consider an obstacle whose boundary is described by a smooth function* $\Gamma(\boldsymbol{q}) = 0$, *where* $\Gamma : \mathbb{R}^d \to \mathbb{R}$ *is a continuously differentiable function* $(\mathcal{C}^1)$. *Obstacle boundary normal is represented as* $\boldsymbol{n}(\boldsymbol{q}) = \nabla\Gamma(\boldsymbol{q})/\|\nabla\Gamma(\boldsymbol{q})\|_2$. *A vector field* $\boldsymbol{f}(\boldsymbol{q})$ *is not penetrating the obstacle boundary if its projection onto the obstacle normal* $\boldsymbol{n}(\boldsymbol{q})$ *vanishes on the obstacle's surface* $\Gamma(\boldsymbol{q}) = 0$*:*

$$\boldsymbol{f}(\boldsymbol{q})^T\boldsymbol{n}(\boldsymbol{q}) = 0 \quad \forall \boldsymbol{q} : \Gamma(\boldsymbol{q}) = 0. \tag{8}$$

Definition 1 is derived from (Khansari-Zadeh and Billard, 2012). Nonetheless, this definition is only valid for the continuous case, whereas any robotics application necessitates discretizing the dynamics. Specifically, even for the case of convex obstacles, if the integration timestep δt is sufficiently large, a single iteration could bring the system state inside the obstacle. For concave obstacles, an additional mode of failure arises, in which any motion in the tangential plane will violate the obstacle boundary if the system starts on the obstacle surface. We propose a more general definition of impenetrability for the discretized case. Notably, it is still applicable for continuous dynamics.

Definition 2. (Impenetrability). *Consider an obstacle whose boundary is described by isosurface* $\Gamma(\boldsymbol{q}) = 0$, *where* $\Gamma : \mathbb{R}^d \to \mathbb{R}$. *A vector field* $\boldsymbol{f}(\boldsymbol{q})$ *is not penetrating the obstacle boundary, if for any trajectory* $\{\boldsymbol{q}\}_t$ *that starts outside the obstacle boundary,* that is, $\Gamma(\boldsymbol{q}_0) \geq 0$, *and evolving according to* $\dot{\boldsymbol{q}} = \boldsymbol{f}(\boldsymbol{q})$, *the following holds:* $\Gamma(\{\boldsymbol{q}\}_t) \geq 0, \forall t$.

Definition 1 is useful for analyzing the stability properties of modulation. Spurious attractors, which can be can be local minima or saddle points, are induced when the nominal DS and obstacle normal are collinear at the obstacle boundary, that is, $|\langle \boldsymbol{f}(\boldsymbol{q}), \boldsymbol{n}(\boldsymbol{q})\rangle| = \|\boldsymbol{f}(\boldsymbol{q})\|$ and $\langle \boldsymbol{f}(\boldsymbol{q}), \boldsymbol{e}_i(\boldsymbol{q})\rangle = 0$ holds for all i. Refer to Figure 4 for demonstration of the local minima created with a simple concave obstacle.

4.2. Convergence for star-shaped obstacles

An improvement upon the modulation framework for obstacle avoidance to avoid spurious attractors and preserve stability is proposed by Huber et al. (2019, 2022). Instead of using the obstacle normal $\boldsymbol{n}(\boldsymbol{q})$ as the first column of $\boldsymbol{E}(\boldsymbol{q})$, the authors propose to instead use the reference direction $\widehat{\boldsymbol{r}}(\boldsymbol{q}, \boldsymbol{q}^r)$ defined as

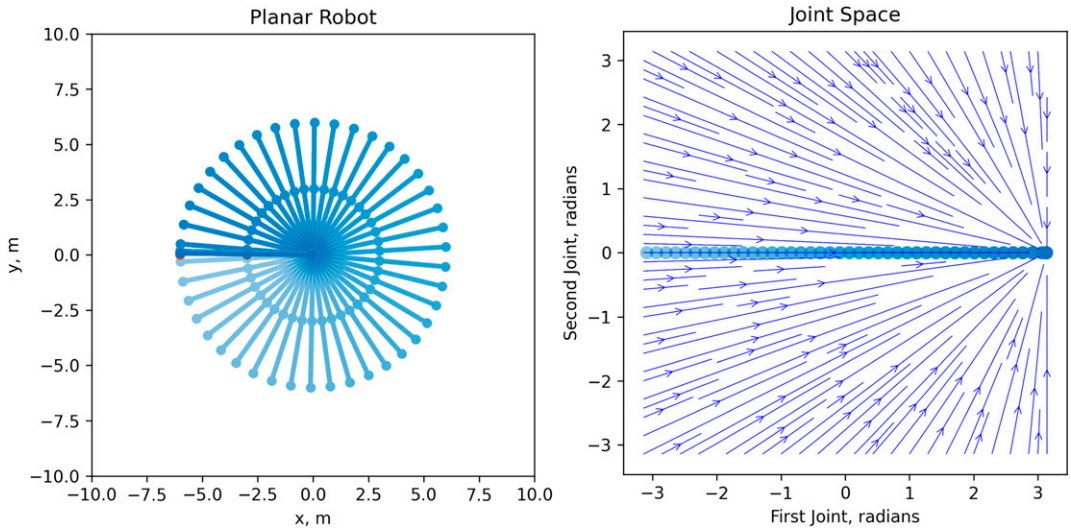


Figure 2. Nominal linear DS motion (equation (1)) of a planar 2-DoF robot. The robot sweeps through the workspace in the absence of obstacles. The task space is visualized on the left, while the joint space is on the right.

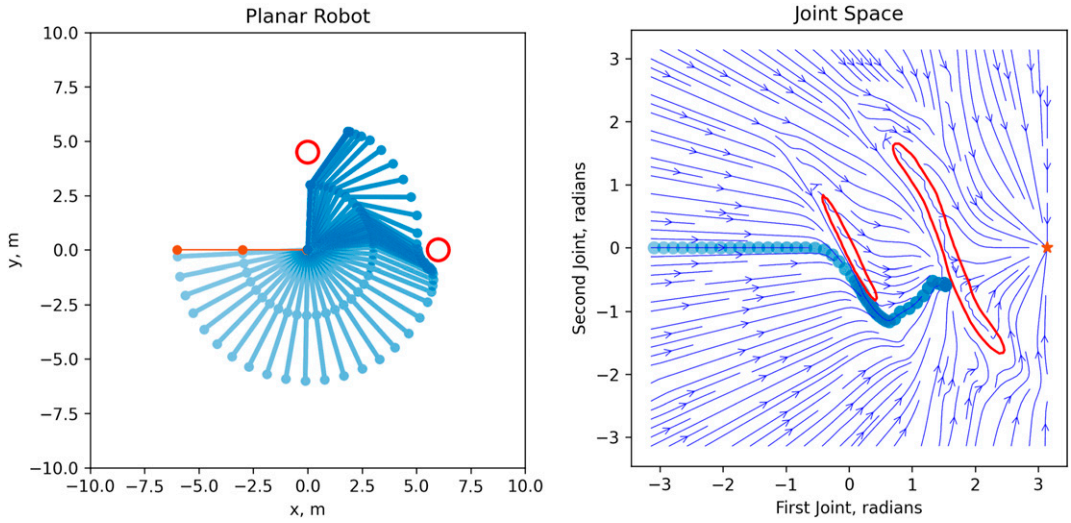


Figure 3. Modulated DS motion (equation (4)) of a planar 2-DoF robot in presence of two circular obstacles (in red). The robot avoids the first obstacle but gets stuck near the second due to the vanishing tangential component after modulation.

$$\widehat{\boldsymbol{r}}(\boldsymbol{q},\boldsymbol{q}^r)=\frac{\boldsymbol{q}^r-\boldsymbol{q}}{\|\boldsymbol{q}^r-\boldsymbol{q}\|}, \tag{9}$$

where $\boldsymbol{q}^r$ is a point belonging to the obstacle.

While discarding the orthonormality of the basis $\boldsymbol{E}(\boldsymbol{q})$, this modification allows for the system to avoid obstacles with non-convex boundaries that can be represented as star-shaped obstacles. The reference direction $\widehat{\boldsymbol{r}}(\boldsymbol{q},\boldsymbol{q}^r)$ is not orthogonal to any of the basis vectors $\boldsymbol{e}_i$, thus $\langle\widehat{\boldsymbol{r}}(\boldsymbol{q},\boldsymbol{q}^r),\boldsymbol{f}(\boldsymbol{q})\rangle$ is projected onto tangential space, and there exists a tangential vector flow on the obstacle boundary, alleviating the local minima issue for any point on the boundary expect for a single saddle point when $\langle\widehat{\boldsymbol{r}}(\boldsymbol{q},\boldsymbol{q}^r),\boldsymbol{f}(\boldsymbol{q})\rangle$ are collinear.

Interestingly, the improved modulated DS approach using reference direction (9) to construct $\boldsymbol{E}(\boldsymbol{q})$ can be interpreted as a nonlinear control system with the following form:

$$\dot{\boldsymbol{q}}=\underbrace{\boldsymbol{M}(\boldsymbol{q})\cdot\boldsymbol{f}(\boldsymbol{q})}_{\text{ModulatedDynamics}}+\underbrace{\boldsymbol{g}(\boldsymbol{q},\boldsymbol{q}^r)}_{\text{Input}}, \tag{10}$$

where $\boldsymbol{g}(\boldsymbol{q},\boldsymbol{q}^r)$ is a nonlinear function that depends on the current state $\boldsymbol{q}$ and the obstacle reference point $\boldsymbol{q}^r$ and pushes the robot state away from local minima. This interpretation connects the DS modulation approach to optimal control domain, as the tangent space vector flow can be thought of as a virtual control input applied to the system. Thus, $\boldsymbol{q}^r$ can be considered a control variable that is forcing the modulation to avoid $\boldsymbol{M}(\boldsymbol{q})\cdot\boldsymbol{f}(\boldsymbol{q})\to 0$ at the obstacle boundary.

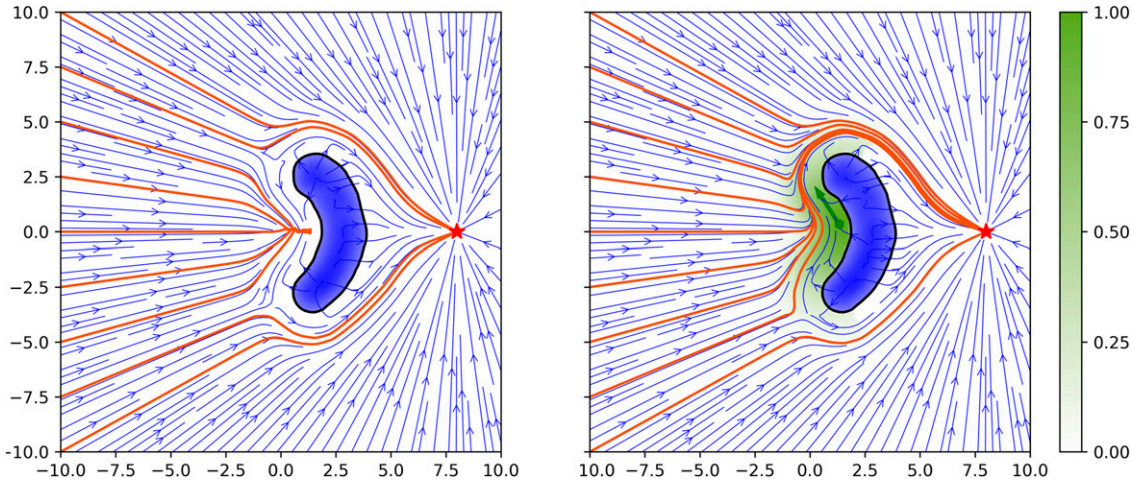


Figure 4. Two-dimensional toy example demonstrating the behavior of the modulated DS in presence of obstacle (blue). The system is linear with stable attractor (red) at (8,0). Orange lines indicate trajectories integrated forward in time. **(left)** Modulated DS (Khansari-Zadeh and Billard, 2012) has local minima in the concave region of the obstacle. **(right)** Our method avoids the local minima by adding one navigation kernel with parameters estimated via sampling-based MPC. Green arrow indicates the center $\widehat{q}_k$ and the direction $\widehat{g}_k$ of the added deflection. Green shaded area shows the kernel activation region.

While (10) alleviates the creation of spurious attractors by inducing additional tangent space velocities through the additive nonlinear function $\boldsymbol{g}(\boldsymbol{q}, \boldsymbol{q}^r)$, such term is difficult to control and scale to high-dimensional problems. Specifically, $\boldsymbol{g}(\boldsymbol{q}, \boldsymbol{q}^r)$ is dependent on the projection of the nominal DS $\boldsymbol{f}(\boldsymbol{q})$ and the placement of reference point $\boldsymbol{q}^r$. Finding a suitable $\boldsymbol{q}^r$ is not trivial for high-dimensional concave obstacles, thus it may be challenging to build an algorithm that guarantees the collision avoidance and convergence in case of multiple complex obstacles. Additionally, this method introduces the need to invert the matrix $\boldsymbol{E}(\boldsymbol{q})$ at every iteration, which can be computationally expensive for high-frequency applications in high-dimensional systems.

5. Locally deflected obstacle-tangent space dynamics

Inspired by the state-input system interpretation introduced in (10), we present our approach for creating meaningful obstacle-tangent deflections that are locally active in the regions of potential local minima of the modulated nominal DS. To alleviate the shortcomings of (Huber et al., 2019; Khansari-Zadeh and Billard, 2012), we propose the following formulation for modulating the DS:

$$\begin{aligned}\dot{\boldsymbol{q}} &= \boldsymbol{M}(\boldsymbol{q})(\boldsymbol{f}(\boldsymbol{q}) + \alpha(\boldsymbol{q})\boldsymbol{g}(\boldsymbol{q})) \\ &= \underbrace{\boldsymbol{M}(\boldsymbol{q}) \cdot \boldsymbol{f}(\boldsymbol{q})}_{\text{Standard Modulation}} + \underbrace{\alpha(\boldsymbol{q})}_{\in[0,1]} \underbrace{\boldsymbol{M}(\boldsymbol{q}) \cdot \boldsymbol{g}(\boldsymbol{q})}_{\text{Explicit Tangent Space Dynamics}}\end{aligned} \tag{11}$$

where $\boldsymbol{M}(\boldsymbol{q}) \in \mathbb{R}^{d \times d}$ is a modulation matrix as defined in equations (5)–(7), and (13), $\boldsymbol{f}(\boldsymbol{q}) : \mathbb{R}^d \to \mathbb{R}^d$ is the nominal DS, and $\boldsymbol{g}(\boldsymbol{q}) : \mathbb{R}^d \to \mathbb{R}^d$ is a vector field that is explicitly defined to be locally tangential to the obstacle boundary, that is, $\boldsymbol{g}(\boldsymbol{q}) \perp \boldsymbol{n}(\boldsymbol{q})$, with $\boldsymbol{n}(\boldsymbol{q})$ being the obstacle normal.

Continuous scalar-valued activation function $\alpha(\boldsymbol{q}) : \mathbb{R}^d \to \mathbb{R}$ indicates the regions in state space where the obstacle-tangential vector field $\boldsymbol{g}(\boldsymbol{q})$ is active; with $\alpha(\boldsymbol{q}) \in [0, 1]$. When the nominal modulated DS term is close to local minima (i.e., $\boldsymbol{M}(\boldsymbol{q})\cdot\boldsymbol{f}(\boldsymbol{q}) \to 0$), then $0 < \alpha(\boldsymbol{q}) \leq 1$, otherwise, $\alpha(\boldsymbol{q}) = 0$. Obstacle-tangential velocities $\boldsymbol{g}(\boldsymbol{q})$ are thus introduced only to avoid generation of spurious attractors. Figure 4 demonstrates the possible deflection added in the vicinity of local minima.

The advantage of (11) over (Huber et al., 2019; Khansari-Zadeh and Billard, 2012;) is that $\boldsymbol{g}(\boldsymbol{q})$ is independent of $\boldsymbol{f}(\boldsymbol{q})$ or any reference point $\boldsymbol{q}^r$, opening the possibility of defining such tangential vector field with external optimization techniques analogous to a stabilizing optimal control input. This allows an intuitive and explicit control over the tangential component of the vector field yielding an inherent boundary impenetrable modulated DS.

5.1. Distance function adaptation

Our goal is to use a learned model of the true distance-to-collision, as defined in (3), to define the collision boundary. To preserve the collision avoidance properties of the modulation $\boldsymbol{M}(\boldsymbol{q})$ defined in equation (5), we need to properly define the functions $\lambda_n(\boldsymbol{q})$ and $\lambda_\tau(\boldsymbol{q})$, which are used to construct the diagonal gains matrix $\boldsymbol{D}(\boldsymbol{q})$.

We employ a parametrized sigmoid function:

$$\begin{aligned}&\sigma(d_1, d_2, \lambda_1, \lambda_2, k) = \\ &= \lambda_1 + \frac{\lambda_2 - \lambda_1}{1 + \exp\left(-k\left(\Gamma(\boldsymbol{q}) - \frac{(d_1 + d_2)}{2}\right)\right)},\end{aligned} \tag{12}$$

which smoothly connects the constant values λ_1 and λ_2 as a function of distance $\Gamma(\boldsymbol{q})$. The parameter k controls the width of the transition, and the offsets d_1 and d_2 define the transition mid-point.

To define $\lambda_n(\boldsymbol{q})$, we use $\lambda_1 = 0$ and $\lambda_2 = 1$, while for $\lambda_\tau(\boldsymbol{q})$, $\lambda_1 = 2$ and $\lambda_2 = 1$. Distance parameters d_1 and d_2 are set to be 0 *cm* and 10 *cm*, respectively, and parameter k is set to $k = 2$. Thus, diagonal elements of matrix $\boldsymbol{D}(\boldsymbol{q})$ are defined as

$$\begin{aligned} \lambda_n(\boldsymbol{q}) &= \sigma(1cm, 10cm, 0, 1, 2) \\ \lambda_\tau(\boldsymbol{q}) &= \sigma(1cm, 10cm, 2, 1, 2) \end{aligned} \tag{13}$$

With such construction, $\lambda_n(\boldsymbol{q}) = \lambda_\tau(\boldsymbol{q}) = 1$ (making matrix $\boldsymbol{D}(\boldsymbol{q})$ to be identity) for $\Gamma(\boldsymbol{q}) > 10$ cm. For situations when distance-to-collision is lower than threshold $\Gamma(\boldsymbol{q}) = 1$ cm, $\lambda_n(\boldsymbol{q}) = 0$ and $\lambda_\tau(\boldsymbol{q}) = 2$, guaranteeing non-penetrability. Following such definition, the non-negative nature of the values λ_n and λ_τ ensures that the matrix M is positive definite. Consequently, M cannot reverse the nominal DS flow; it can only redirect it. However, the incorporation of additional tangent dynamics enables the generation of motion that can oppose the direction of the original DS.

Proposition 1. Impenetrability. *Consider an obstacle in the task space of the robot, represented by a set of points* $\subset \mathbb{R}^3$. *Let* $\Gamma(\boldsymbol{q}) : \mathbb{R}^d \rightarrow \mathbb{R}$ *be a continuous and continuously differentiable* ($\mathcal{C}^1$) *function that expresses the minimal distance between the robot in state* $\boldsymbol{q}$ *and set* $\mathcal{O}$. *System* (11) *preserves the impenetrability of the obstacle*.
Proof: *See* Appendix B.

5.2. Explicit tangent space dynamics as navigation kernels

We propose to construct the vector field $\boldsymbol{g}(\boldsymbol{q})$ that will be projected onto the tangent space of the obstacle boundary as a sum of K local dynamics $\widehat{\boldsymbol{g}}_k$, which we refer to as *navigation kernels*, activated by radial-basis functions as follows:

$$\boldsymbol{g}(\boldsymbol{q}) = \sum_{k=1}^{K} \widehat{\boldsymbol{g}}_k \exp(-\gamma_k \|\boldsymbol{q} - \widehat{\boldsymbol{q}}_k\|), \tag{14}$$

where $\widehat{\boldsymbol{q}}_k$ is the centroid of k-th navigation kernel, and γ_k regulates kernel width. Vector $\widehat{\boldsymbol{g}}_k$ is a linear local dynamics activated in the vicinity of the kernel center $\widehat{\boldsymbol{q}}_k$.

Essentially, formulation (14) allows $\boldsymbol{g}(\boldsymbol{q})$ to be defined by a set of K navigation kernels, so that $\boldsymbol{g}(\boldsymbol{q})$ is present when the state $\boldsymbol{q}$ is close to the k-th kernel center $\widehat{\boldsymbol{q}}_k$, and exponentially decays as the distance to the kernel center increases. If kernels are placed close to the obstacles (so that $\Gamma(\widehat{\boldsymbol{q}}_k) < \varepsilon$), then the added tangential modulation will be stronger in the vicinity of the obstacle, and will decay as the distance to the obstacle increases.

5.3. Navigation kernel activation

The navigation kernels defined in (14) are activated in the deflected dynamics (11) by a state-dependent scalar-valued function $\alpha(\boldsymbol{q}) \in [0, 1]$, that is used to reduce the effect of the vector field $\boldsymbol{g}(\boldsymbol{q})$ when the system is not in the local minima induced by the obstacle. This function consists of three components:

$$\alpha(\boldsymbol{q}) = \begin{cases} \alpha_\Gamma(\boldsymbol{q}) \cdot \alpha_n(\boldsymbol{q}) \cdot \alpha_f(\boldsymbol{q}), & \|\boldsymbol{q} - \boldsymbol{q}^*\| > r \\ 0, & \|\boldsymbol{q} - \boldsymbol{q}^*\| \leq r \end{cases} \tag{15}$$

where $\alpha_\Gamma(\boldsymbol{q})$ is equal to one when the system is close to the obstacle, and decays to zero with the increase of distance-to-collision. For example, we can define $\alpha_\Gamma(\boldsymbol{q}) = 1 - \lambda_n(\boldsymbol{q})$, rendering navigation kernels inactive for any robot state that is further than 10 cm from collision. Values of $\alpha_n(\boldsymbol{q})$ change between 0 and 1 depending on the angle between the obstacle normal and the nominal DS direction. Finally, $\alpha_f(\boldsymbol{q})$ continuously changes from 1 to 0 as the system state $\boldsymbol{q}$ approaches the nominal DS attractor $\boldsymbol{q}^*$.

Additionally, we enforce $\alpha(\boldsymbol{q}) = 0$ within a ball B_r with radius r enclosing the attractor $\boldsymbol{q}^*$, to be able to preserve local stability of the attractor. Refer to Figure 4 for visualization of active area for a kernel placed close to the concave obstacle. Exact mathematical definitions for $\alpha_{\{\Gamma, n, f\}}$ are provided further in Section 7.2.

Proposition 2. (L.A.S). *Consider a modulated DS in the form* $\dot{\boldsymbol{q}} = \boldsymbol{M}(\boldsymbol{q})\boldsymbol{f}(\boldsymbol{q})$. *Assume that* $\boldsymbol{f}(\boldsymbol{q})$ *is G.A.S. at the attractor* $\boldsymbol{q}^*$, *modulation matrix* $\boldsymbol{M}(\boldsymbol{q})$ *has full rank* $\forall \boldsymbol{q} \in \mathbb{R}^d$ *and is equal to an identity matrix in a ball* B_r *with radius r centered at* $\boldsymbol{q}^*$, that is, $\boldsymbol{M}(\boldsymbol{q}) = \mathbb{I}_{d \times d}$, $\forall \boldsymbol{q}$: $\boldsymbol{q} \in B_r$. *The modulated DS with the tangential deflection component (as in equations* 11, 14, *and* 15) *in a form* $\dot{\boldsymbol{q}} = \boldsymbol{M}(\boldsymbol{q})(\boldsymbol{f}(\boldsymbol{q}) + \alpha(\boldsymbol{q})\boldsymbol{g}(\boldsymbol{q}))$ *is locally asymptotically stable (L.A.S.) at the attractor* $\boldsymbol{q}^*$ *if the activation parameter* $\alpha(\boldsymbol{q}) = 0$, $\forall \boldsymbol{q}$: $\boldsymbol{q} \in B_r$.
Proof: See Appendix C.

5.4. Navigation kernel parameter optimization

The combination of K kernels allows to define a policy that is locally active, and is able to provide meaningful tangential components to the nominal DS enabling navigation around obstacles. The described policy is defined by direction $\widehat{\boldsymbol{g}}_k$, kernel center $\widehat{\boldsymbol{q}}_k$, and kernel width γ_k for each k-th kernel, respectively. To determine these parameters, we propose to use MPPI approach similar to (Bhardwaj et al., 2022).

The MPPI algorithm provides a framework for finding the optimal parameters of a navigation policy by sampling a distribution of system parameters and then evaluating the corresponding cost of each trajectory generated by these samples. In our proposed formulation, we apply MPPI to learn the parameters of the navigation kernels. Specifically, the MPPI algorithm will be used to search over the space of $\widehat{\boldsymbol{g}}_k$, which represents the local dynamics associated with each kernel.

6. Navigation kernel parameters optimization via sampling-based MPC (MPPI)

Section 6.1 provides an introduction to the classical MPPI method. Subsequent Sections 6.2-6.4 detail how we utilize MPPI to determine the optimal parameters for navigation kernels introduced previously.

6.1. MPPI algorithm description

The idea behind the MPPI method is to sample the system parameters from a distribution, and then to use the sampled parameters to generate a set of trajectories. The cost of each trajectory is computed, and used to change the distribution and bias the sampling procedure towards the parameters that lead to the lower cost. Eventually, the procedure converges to parameters that minimize the defined cost function (Yoon et al., 2022).

Let's consider an autonomous discrete-time Dynamical System of the form

$$\boldsymbol{q}_h = \tilde{\boldsymbol{f}}(\boldsymbol{q}_{h-1}, \boldsymbol{g}_1, \ldots, \boldsymbol{g}_K), \tag{16}$$

where $\boldsymbol{q}_h$ is the state vector at time-index h and parameters $\boldsymbol{g}_{k=1\ldots K}$ alter the system's behavior. If $\boldsymbol{g}_k = 0$, $\forall k$, then the system behaves as some nominal DS.

At each iteration, N candidate parameter sets $\{\boldsymbol{g}_{i,k}\}_{i=1..N}^{k=1..K}$ are sampled from K independent multivariate Gaussian distributions such that $\boldsymbol{g}_{i,k} \sim \mathcal{N}_k(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$, and then used to generate N trajectories by propagating system (16) forward in time for H timesteps. We denote resulting trajectories (or *roll-outs*) as $\{\boldsymbol{q}_{i,h}, \dot{\boldsymbol{q}}_{i,h}\}_{i=1..N}^{h=1..H}$. After that, the cost for each trajectory is defined as $c_i = c\ (\boldsymbol{q}_{i,1}, \ldots, \boldsymbol{q}_{i,H})$. The cost is designed according to the desired behavior, that is, to prefer trajectories that are close to the nominal DS, or to enable obstacle avoidance, or to be compliant with secondary tasks. Corresponding weights are then calculated as

$$w_i = \exp\left(-\frac{c_i}{\beta}\right), \tag{17}$$

and the means of the parameter distributions are updated as follows:

$$\boldsymbol{\mu}_k^{new} = (1-\alpha_\mu)\boldsymbol{\mu}_k + \alpha_\mu \frac{\sum_{i=1}^N w_i \boldsymbol{g}_{i,k}}{\sum_{i=1}^N w_i}, \tag{18}$$

essentially shifting the distribution center towards the parameters that lead to the lowest cost. The covariance matrix may also be updated:

$$\boldsymbol{\Sigma}_k^{new} = (1-\alpha_\Sigma)\boldsymbol{\Sigma}_k + \\ + \alpha_\Sigma \frac{\sum_{i=1}^N w_i (\boldsymbol{g}_{i,k} - \boldsymbol{\mu})(\boldsymbol{g}_{i,k} - \boldsymbol{\mu}_k)^T}{\sum_{i=1}^N w_i}. \tag{19}$$

After updating distributions parameters with $\boldsymbol{\mu}_k^{new}$ and $\boldsymbol{\Sigma}_k^{new}$, each parameter $\boldsymbol{g}_k$ is chosen either as the one with lowest corresponding cost c_i, or as a weighted sum (last term in (18)), or as $\boldsymbol{\mu}_k^{new}$, and the system is propagated forward in time for one timestep, and the process is repeated.

In (18)-(19), α_μ and α_Σ are the learning rates for the mean and covariance matrix, respectively. The parameter $\beta > 0$ is a temperature parameter that controls the amount of exploration. The higher the temperature, the more the distribution is spread out, and the more the exploration is performed. Williams et al. (2017) prove that such process iteratively converges to the optimal parameters that minimize the cost function.

6.2. MPPI application

Navigation vector field $\boldsymbol{g}(\boldsymbol{q})$ is defined as sum of locally active navigation kernels (14), and each kernel is defined by three parameters: local deflection direction $\widehat{\boldsymbol{g}}_k$, kernel placement $\widehat{\boldsymbol{q}}_k$ and kernel width γ_k.

We propose to utilize the MPPI framework to determine the local navigation direction $\widehat{\boldsymbol{g}}_k$ of each kernel. Additionally, numerous exploration trajectories produced by the MPPI algorithm at each iteration may be used to determine the placement of the navigation kernels, that is, values of $\widehat{\boldsymbol{q}}_k$.

While kernel widths γ_k can also be optimized via MPPI, for simplicity we fix $\gamma_k = \widehat{\gamma}$, $\forall k$ as a hyperparameter. Kernel placement $\widehat{\boldsymbol{q}}_k$ is determined during exploration using the roll-out states. The only parameter set to find via MPPI is then $\{\widehat{\boldsymbol{g}}_k\}_{k=1..K}$. Such approach enables exploration of a large range of potential navigation strategies in the vicinity of the obstacle.

At each iteration of the MPPI algorithm, parameter set $\{\widehat{\boldsymbol{g}}_{i,k}\}_{i=1..N}^{k=1..K}$ is sampled from K multivariate Gaussian distributions that are parameterized by $\{\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1..K}$. We then use the sampled parameters to generate N roll-outs $\{\boldsymbol{q}_{i,h}, \dot{\boldsymbol{q}}_{i,h}\}_{i=1..N}^{h=1..H}$ by propagating the system dynamics H timesteps forward in time.

Next, the cost of each trajectory is computed, which is then used to bias the sampling procedure to achieve a lower cost. The weights for each sample are calculated using equation (17), and the mean of the parameter distributions is updated using equation (18). Once the new sampling distribution has been determined, we update the system state, sample new sets of parameters, and the process repeats.

An example of a placed navigation kernel with parameters optimized using MPPI is demonstrated in Figure 4. Notably, in this two-dimensional toy example, the navigation strategy is relatively straightforward—track the obstacle boundary in either the left or right direction. However, in three dimensions, the tangential plane consists of an infinite number of directions to track the obstacle boundary. As dimensionality increases, it becomes exponentially more challenging to find valid navigation strategies. We apply the MPPI algorithm to a 7-DoF robot arm, where an infinite number of navigation strategies can be explored. Examples of successful navigation strategies for such cases are illustrated in Figures 1 and 11.

6.3. Cost function

At each iteration of the MPPI algorithm, a cost for each roll-out $\{\boldsymbol{q}_{i,h}, \dot{\boldsymbol{q}}_{i,h}\}_{i=1..N}^{h=1..H}$ is computed. The cost function encodes desired robot behavior, and is used to bias the sampling procedure towards the parameters that minimize the cost. Cost function can be defined as a weighted sum of several cost terms. In this work, we consider the following cost terms:

6.3.1. Goal reaching. The first cost term is a goal reaching cost, which is defined as follows:

$$c_i^{goal} = \left\| \boldsymbol{q}^* - \boldsymbol{q}_{i,H} \right\|_2, \tag{20}$$

where $\boldsymbol{q}^*$ is the attractor of the nominal DS (1), and $\boldsymbol{q}_{i,H}$ is the position of the robot at the end of the i-th roll-out.

We only consider the final position of the roll-out, as sampled trajectory may temporarily move away from the goal while navigating around the obstacle. This cost penalizes trajectories that do not approach to the goal in the integration horizon H.

6.3.2. (Self-)Collision avoidance. Distance estimator $\Gamma(\boldsymbol{q})$ provides true distance between the robot and the closest obstacle. As the intended behavior of the motion planner is to navigate around the obstacles, a continuous function penalizing close proximity to the obstacles may restrict the exploration of the space of possible navigation strategies. Therefore, we use a binary collision detection function that penalizes only trajectories that collide with the obstacle. The collision avoidance cost is defined as follows:

$$c_i^{coll} = \sum_{h=1}^{H} c_{i,h}^{coll}, \tag{21}$$

where

$$c_{i,h}^{coll} = \begin{cases} 0, & \text{if } \Gamma(\boldsymbol{q}_{i,h}) > 0 \\ 1, & \text{otherwise.} \end{cases} \tag{22}$$

For self-collision avoidance cost $c_i^{self-coll}$, we use the same cost function, but with the distance estimator $\Gamma(\boldsymbol{q})$ replaced by $\Gamma_{self}(\boldsymbol{q})$.

6.3.3. Joint limits avoidance. We treat joint limits violation in the same way as collisions. Each state is penalized for violation of the joint limits, and the cost is defined as follows:

$$c_i^{\lim} = \sum_{h=1}^{H} c_{i,h}^{\lim}, \tag{23}$$

where

$$c_{i,h}^{\lim} = \begin{cases} 0, & \text{if } \boldsymbol{q}_{i,h} \in [\boldsymbol{q}_{min}, \boldsymbol{q}_{max}] \\ 1, & \text{otherwise.} \end{cases} \tag{24}$$

Since we assume the DS control in the joint space, we are not concerned by low-manipulability issues, thus do not dampen the joints close to joint-limits.

6.3.4. Stagnation avoidance. The cost function also includes a term that penalizes trajectories that do not move with time. This cost is defined as follows:

$$c_i^{stay} = \frac{c_i^{goal}}{\max\left(\varepsilon, \left\| \boldsymbol{q}_{i,1} - \boldsymbol{q}_{i,H} \right\|_2\right)}. \tag{25}$$

This cost encourages the exploration, while penalizing the stagnant trajectories away from the attractor, thus helping to avoid getting stuck in local minima. To some extent, this cost also acts against appearance of limit cycles, that are theoretically possible for the DS modulation approach. However, the length of a cycle should correspond to the horizon of the MPC path prediction. As the trajectories converge to $\boldsymbol{q}^*$, this cost becomes zero.

6.3.5. Nominal DS similarity. Another cost component we propose to use measures the difference between the nominal DS and the actual trajectory. This cost is defined as follows:

$$c_i^{DS} = \sum_{h=1}^{H} \Gamma(\boldsymbol{q}_{i,h}) \left\| \dot{\boldsymbol{q}}_{i,h} - f(\boldsymbol{q}_{i,h}) \right\|_2, \tag{26}$$

and allows the robot to deviate from the nominal vector field when close to the obstacle.

6.3.5.1. Total cost. The total cost for each trajectory is defined as a weighted sum of the cost terms defined above:

$$c_i = \sum_{t \in T} w_t c_i^t, \tag{27}$$

where w_t is the weight of the cost term t, and T is the set of cost terms $T = \{goal, coll, self-coll, \lim, stay, DS\}$.

6.4. New kernel placement

MPPI algorithm is essentially a sampling-based motion planner generating new explorative trajectories at each iteration. For each point $\boldsymbol{q}_{i,h}$ at runtime the distance-to-collision $\Gamma(\boldsymbol{q}_{i,h})$, local obstacle repulsion $\boldsymbol{n}$ and the distance-to-goal $\|\boldsymbol{q}^* - \boldsymbol{q}_{i,h}\|_2$ are evaluated. We may use this exploration to better place the navigation kernels, and add them dynamically as new obstacles are detected along the sampled trajectories.

If stable region of the nominal DS attractor is not reached yet, the distance-to-collision is less than a threshold parameter, and system is within the region of possible local minima a new navigation kernel can be placed. Local minima detection can be characterized as scalar product value $\langle \boldsymbol{n}(\boldsymbol{q}), \widehat{f}(\boldsymbol{q}) \rangle$ being close to -1, where $\widehat{f}(\boldsymbol{q}) = \boldsymbol{f}(\boldsymbol{q}) / \|\boldsymbol{f}(\boldsymbol{q})\|$ is a normalized direction of nominal DS, and $\boldsymbol{n}(\boldsymbol{q})$ is obstacle normal. Additionally we consider the proximity to the existing set of K navigation kernels. Overall, the criteria for placing a new navigation kernel are

$$\begin{aligned} \{\widehat{\boldsymbol{q}}_k\}_{k=1}^{K+1} &= \boldsymbol{q}_{i,h} \cup \{\widehat{\boldsymbol{q}}_k\}_{k=1}^{K} \\ \Updownarrow & \end{aligned} \tag{28a}$$

$$\begin{cases} \left\|\boldsymbol{q}^* - \boldsymbol{q}_{i,h}\right\|_2 > \delta_*, & \Leftrightarrow \text{ Far from the nominal attractor.} \\ \Gamma(\boldsymbol{q}_{i,h}) < \delta_\Gamma, & \Leftrightarrow \text{ Close to collision with obstacle.} \\ \left\langle \boldsymbol{n}(\boldsymbol{q}), \widehat{f}(\boldsymbol{q}) \right\rangle < -1 + \delta_n, & \Leftrightarrow \text{ Possible local minima for standard modulation.} \\ \min_{k=1..K} \left\|\boldsymbol{q}_{i,h} - \widehat{\boldsymbol{q}}_k\right\|_2 > \delta_k. & \Leftrightarrow \text{ Far from existing navigation kernels.} \end{cases} \tag{28b}$$

Parameters δ_*, δ_Γ, δ_n, δ_k are strictly positive and their values determine the density of the navigation kernels placed during exploration. Notably, some conditions of kernel placing duplicate the mechanisms of kernel activations α_Γ, α_n and α_f. In other words, even if a kernel is placed close to attractor $\boldsymbol{q}^*$, it would not get activated, because $\alpha_f = 0$. However, such redundancy provides more tunability for the system, and reduces the overall amount of navigation kernels placed.

After the new candidate for a navigation kernel is found, it is added to the existing set of kernels, centered at $\widehat{\boldsymbol{q}}_{k+1} = \boldsymbol{q}_{i,h}$, has kernel width $\gamma_k = \widehat{\gamma}$, and tangential direction $\boldsymbol{g}_{k+1} = \boldsymbol{\mu}_{k^*}$, where k^* stands for the index of closest existing kernel: $k^* = \min_k \left\|\boldsymbol{q}_{i,h} - \widehat{\boldsymbol{q}}_k\right\|_2$. This efficiently warmstarts the optimization of kernel parameters with the parameters of the closest existing kernel.

The sampling policy is also changed to include the new mean $\boldsymbol{\mu}_{k+1} = \boldsymbol{\mu}_{k^*}$ and covariance $\boldsymbol{\Sigma}_{k+1} = \boldsymbol{\Sigma}_{k^*}$. We preserve the similar parameters between closest navigation kernels to avoid undesired oscillatory trajectories. The described MPPI method is detailed in Algorithm 1. The two-dimensional planar robot application is shown in Figure 5.

7. Implementation details

7.1. Tail effect compensation

Modulation $\boldsymbol{M}(\boldsymbol{q})$ defined in equation (5) reshapes the vector field according to defined obstacle basis $\boldsymbol{E}(\boldsymbol{q})$ and weights matrix $\boldsymbol{D}(\boldsymbol{q})$. This modulation only depends on the distance-to-collision $\Gamma(\boldsymbol{q})$ and does not take into account the direction of the vector field. This can lead to a situation where the robot is slowed down by the modulation even if it is moving away from the obstacle. This effect is called the tail effect (Khansari-Zadeh and Billard, 2012). In order to compensate for this effect, we redefine normal direction gain $\lambda_n(\boldsymbol{q})$ as follows:

$$\lambda_n'(\boldsymbol{q}) = \lambda_v(\boldsymbol{q}) + (1 - \lambda_v(\boldsymbol{q}))\lambda_n(\boldsymbol{q}), \tag{29}$$

where $\lambda_v(\boldsymbol{q})$ is a sigmoid function that is defined as

$$\lambda_v(\boldsymbol{q}) = \frac{1}{1 + \exp\left(-100\left\langle \boldsymbol{n}(\boldsymbol{q}), \frac{f(\boldsymbol{q})}{\|f(\boldsymbol{q})\|} \right\rangle\right)}, \tag{30}$$

such that $\lambda_v(\boldsymbol{q})$ continuously indicates whether the nominal vector field is pointing towards the obstacle ($\lambda_v(\boldsymbol{q}) = 0$) or away from it ($\lambda_v(\boldsymbol{q}) = 1$). Thus, $\lambda_n'(\boldsymbol{q})$ is equal to $\lambda_n(\boldsymbol{q})$ if the nominal vector field is pointing towards the obstacle, and is equal to 1 if the nominal vector field is pointing away from the obstacle, even if the distance-to-collision is small. Figure 6 shows the tail compensation effect.

7.2. Navigation kernel activation implementation

For the local activation of navigation kernels, we define parameters $\alpha_\Gamma(\boldsymbol{q})$, $\alpha_n(\boldsymbol{q})$ and $\alpha_f(\boldsymbol{q})$ from equation (15) using the λ_n and λ_v.

$$\begin{aligned} \alpha_\Gamma(\boldsymbol{q}) &= 1 - \lambda_n(\boldsymbol{q}), \\ \alpha_n(\boldsymbol{q}) &= 1 - \lambda_v(\boldsymbol{q}), \\ \alpha_f(\boldsymbol{q}) &= \max(1, \|\boldsymbol{q} - \boldsymbol{q}^*\|). \end{aligned} \tag{31}$$

With such definition, even though navigation kernels are primarily activated by RBFs, they are disabled for situations where distance-to-collision is large, essentially compressing RBFs closer to the obstacle. Additionally, the tail effect is compensated, as $\alpha_n(\boldsymbol{q})$ is equal to 0 if the nominal vector field drives the robot away from the obstacle. Note, that $\alpha_\Gamma(\boldsymbol{q})$ and $\alpha_n(\boldsymbol{q})$ could be merged into a single function using previously defined $\lambda_n'(\boldsymbol{q})$.

7.3. Navigation kernel tangentiality

As stated in Section 5, we define navigation kernel dynamics to be locally tangential to the obstacle boundary. While parameters $\boldsymbol{g}_k$ are sampled from a full d-dimensional Gaussian distribution, we may impose the following transformation on the final policy:

$$\boldsymbol{g}(\boldsymbol{q}) := \boldsymbol{M}^\perp(\boldsymbol{q})\boldsymbol{g}(\boldsymbol{q}), \tag{32}$$

where $\boldsymbol{M}^\perp(\boldsymbol{q})$ is constructed as $\boldsymbol{M}^\perp = \boldsymbol{E}\boldsymbol{D}^\perp\boldsymbol{E}^T$, and $\boldsymbol{D}^\perp$ is an identity matrix with zero as the first element of the first column. With such transformation it is guaranteed that $\boldsymbol{g}(\boldsymbol{q}) \perp \boldsymbol{n}(\boldsymbol{q})$. While this orthogonality is not crucial for obstacle avoidance, as by design any vector field $\boldsymbol{g}(\boldsymbol{q})$ is modulated by $\boldsymbol{M}(\boldsymbol{q})$, it is still a nice property that restricts navigation kernels to only provide meaningful tangential deflection, and not modulate the DS motion in undesired directions.

7.4. Discrete system obstacle impenetrability

Proposition 1 ensures that continuous dynamics defined by (11) do not penetrate the obstacle boundary $\Gamma(\boldsymbol{q}) = 0$.

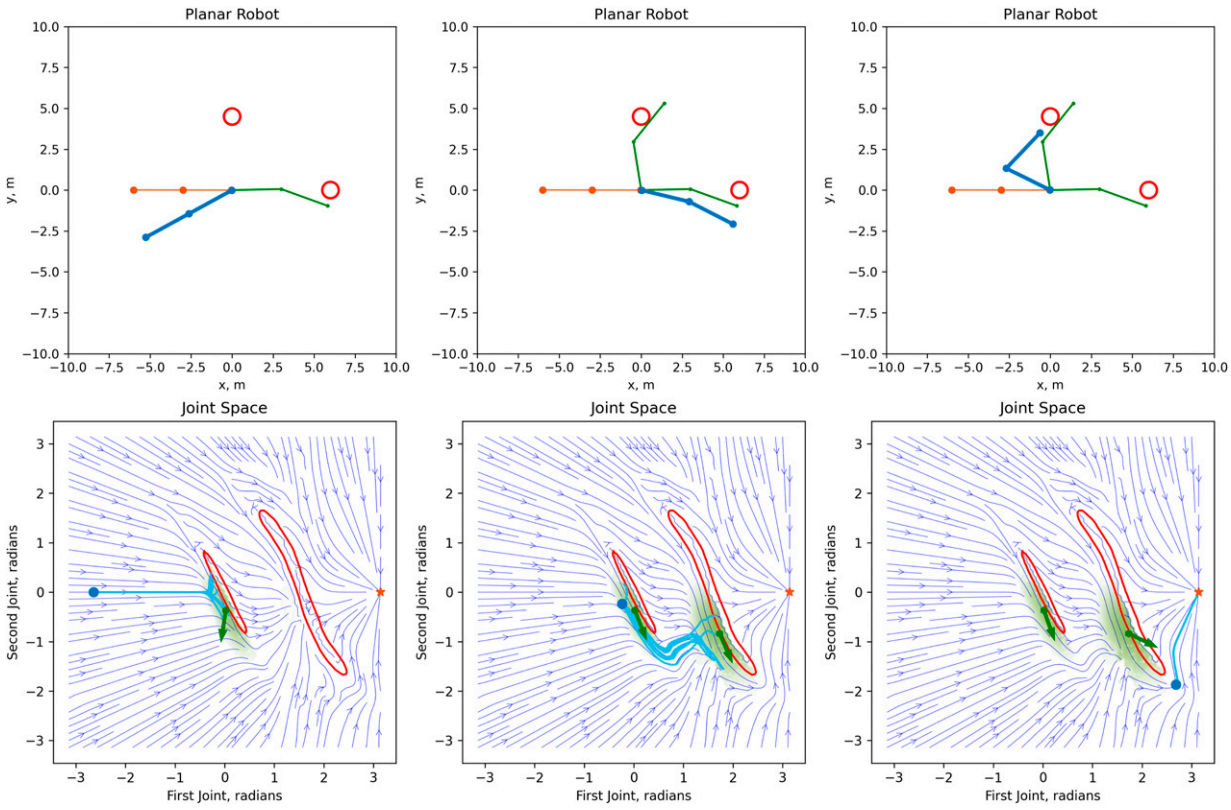


Figure 5. Progression of three states of a planar 2-DoF robot with our algorithm applied. The task space motion is in the top row, while the joint space is visualized at the bottom. By means of sampling of the tangential deflection, a local policy (in green) is generated, allowing the robot to avoid both obstacles and reach the goal. Light blue trajectories show the horizon of MPPI sampling. Figures 2 and 3 demonstrate the corresponding nominal and modulated motions, respectively.

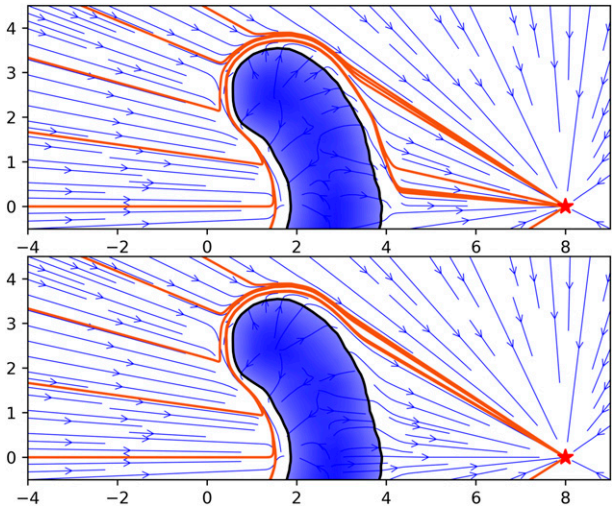


Figure 6. Tail effect visualization: On top is the modulation without tail compensation, that is, using the default λ_n from equation (13). This results in the trajectory "sticking" to the obstacle. Shown below is the modulation with λ_n' introduced in equation (29), which mitigates the "sticking" effect.

However, the Neumann condition is insufficient to guarantee impenetrability for discrete-time dynamics, especially when the integration step is large in comparison to the speed of displacement of the robot, as this can often be the case in real-world robotics applications. Specifically, for convex obstacles, $\boldsymbol{n}(\boldsymbol{q})^T\dot{\boldsymbol{q}}$ on the obstacle boundary does not guarantee impenetrability, as the system can be brought through the obstacle boundary if the timestep δt is large and the previous state is not on the boundary (thus possibly having a nonzero velocity component along the obstacle normal). For concave obstacles, any non-infinitesimal displacement in a tangential plane while on the obstacle boundary within the concavity will also violate the obstacle boundary.

In practical applications, ensuring the non-penetrability of obstacles requires additional considerations. Apart from issues arising from numerical integration, the obstacle's normal is provided by a learned network, which only approximates the normal, thus not guaranteeing impenetrability. To mitigate that, a combination of safety threshold ε, local obstacle repulsion field $\boldsymbol{f}_{\text{rep}}$, and a sufficiently small numerical integration step δt is necessary.

By design, for system (11), $\dot{\boldsymbol{q}} \perp \boldsymbol{n}(\boldsymbol{q})$ for all $\boldsymbol{q}$: $\{\Gamma(\boldsymbol{q}) \leq 1\ cm\}$. In applications with discretized system integration, the robot state may still penetrate the safety threshold $\Gamma(\boldsymbol{q}) = \varepsilon = 1\ cm$. When this occurs, a small repulsive force, $\boldsymbol{f}_{\text{rep}}(\boldsymbol{q}) = k_{\text{rep}}\boldsymbol{n}(\boldsymbol{q})$, is added to the right-hand side of (11) to move the state away from the obstacle boundary and maintain the safety threshold. Note that $\boldsymbol{f}_{\text{rep}}(\boldsymbol{q}) = 0$ for all $\boldsymbol{q}$: $\{\Gamma(\boldsymbol{q}) \geq 1\ cm\}$.

Taking these factors into account, impenetrability for obstacles in discrete system integration can only be violated if a single integration step results in $\Gamma(\boldsymbol{q}_t) \geq \varepsilon$ transitioning to $\Gamma(\boldsymbol{q}_{t+1}) < 0$, or if the repulsion magnitude k_{rep} is insufficient. The first case can be addressed by reducing the numerical integration step δt to an adequately small value. As our algorithm operates at approximately 500 Hz, the value of δt is set to 0.002, ensuring that the integration steps are small enough to prevent overcoming the safety threshold in a single step.

Assuming that the system integration is performed using the Euler method, the magnitude k_{rep} can be estimated by calculating the current undesired penetration through the threshold: $\varepsilon - \Gamma(\boldsymbol{q})$ and introducing the offset with a timestep correction: $k_{\text{rep}} = \frac{\varepsilon - \Gamma(\boldsymbol{q})}{\delta t}$. As a result, the repulsion force added to offset the threshold penetration is defined as

$$\boldsymbol{f}_{\text{rep}}(\boldsymbol{q}) = \begin{cases} \dfrac{\varepsilon - \Gamma(\boldsymbol{q})}{\delta t}\boldsymbol{n}(\boldsymbol{q}), & \forall \boldsymbol{q} : \Gamma(\boldsymbol{q}) < \varepsilon \\ 0, & \text{otherwise.} \end{cases} \tag{33}$$

The threshold $\varepsilon = 1\ cm$ (defined in (13)) in combination with a small timestep $\delta t = 0.002$ ensures impenetrability of the obstacle boundary $\Gamma(\boldsymbol{q}) = 0$ in our practical application.

Overall, the continuous dynamics (11) take the following discrete form:

$$\dot{\boldsymbol{q}}_t = \boldsymbol{M}(\boldsymbol{q}_t)(\boldsymbol{f}(\boldsymbol{q}_t) + \alpha(\boldsymbol{q}_t)\boldsymbol{M}^{\perp}(\boldsymbol{q}_t)\boldsymbol{g}(\boldsymbol{q}_t)) + \boldsymbol{f}_{\text{rep}}(\boldsymbol{q}_t) \tag{34}$$

$$\boldsymbol{q}_{t+1} = \boldsymbol{q}_t + \dot{\boldsymbol{q}}_t \delta t. \tag{35}$$

7.5. Algorithm implementation

Bhardwaj et al. (2022) report frequencies of up to 125 Hz for sampling-based MPC scheme similar to ours; however, it is only possible due to simple integration of the roll-outs, where the state is linear function of the control input. That enables the evaluation of the state sequence by means of lower-triangular integration tensor, achieving parallelization both across multiple timesteps and multiple samples. However, we rely on $\Gamma(\boldsymbol{q})$ evaluation for each timestep for modulation matrix calculation, making the system nonlinear, and restricting parallel computation in time domain. Mainly because of that our MPC is limited to 10-30 Hz depending on number of samples, obstacles and horizon length. The main computation bottleneck of our algorithm is the evaluation of $\Gamma(\boldsymbol{q})$ and $\Delta\Gamma(\boldsymbol{q})$, followed by QR decomposition of obstacle normal to construct the tangential space basis. The latter is not comparable in amount of operations with MLP evaluation; however, it restricts the use of algorithm to CPU.

While the method leverages the parallel execution for multiple exploration samples, and is fully suitable for GPU implementation, we found that QR decomposition of obstacle bases can only be efficient on CPU, thus all setup runs exclusively on CPU. Newer generations of processors with relatively large cache allow quick computation of the MLP, and overall performance is satisfactory. Notably, more traditional methods to evaluate distance-to-collision and repulsive gradient would lead to even more significant slowdowns.

As such frequencies may still not be sufficient for environments where obstacles are not static, we leverage the modular structure and concurrent execution to improve the performance. A simple one-sample one-timestep integration of modulated DS is not as computationally expensive and can be performed with frequencies up to 500 Hz. We asynchronously compute the modulated DS with latest known Modulation Policy, and stream the data to the low-level impedance controller that is executed at 1 kHz. This controller also mitigates potential abrupt changes in the DS-generated commands that may occur due to discontinuities in the distance function between the robot's links and the obstacles represented by spherical primitives. This allows us to achieve real-time performance in dynamic environments.

Importantly, the MPC does not need to operate continuously in the background, using up computational resources. The MPC sampling is activated only if the predicted robot state comes close to navigation kernels, and it is not engaged in situations where there is no risk of local minima induced by obstacles. In practice, the MPC can run asynchronously at all times, but the number of samples can be varied dynamically. Generally, a single roll-out is required to predict the future trajectory and identify local minima. Once the minima is detected and a navigation kernel is placed, the number of samples increases to determine the optimal deflection direction. When the nominal propagated motion encounters no local minima (or navigation kernels), the sample complexity can be reduced back to a single sample for trajectory prediction. This pattern is further explored in Section 8.

Nominal DS is a parameter to both Modulated DS module and MPC module, and obstacles positions are streamed using Optitrack at 120 Hz. Notably, we do not use ROS for streaming, instead we opt for a lightweight setup based on ZeroMQ publisher-subscriber sockets (Hintjens, 2013).

Overall flowchart of the implemented algorithm is shown in Figure 7. All frequencies are reported for Apple Silicon M2 3.7 GHz CPU. The source code for running the presented algorithm in simulation and on a real robot is available online.[3] The algorithm contains a large set of hyperparameters, all of which are introduced and explained in the text. For clarity, we have compiled all meaningful parameters of our method in Table 4 located in Appendix D.

7.6. Computational complexity

The proposed hybrid controller combines two underlying algorithms—DS and MPC. For both algorithms, the major part of computations is related to state propagation,

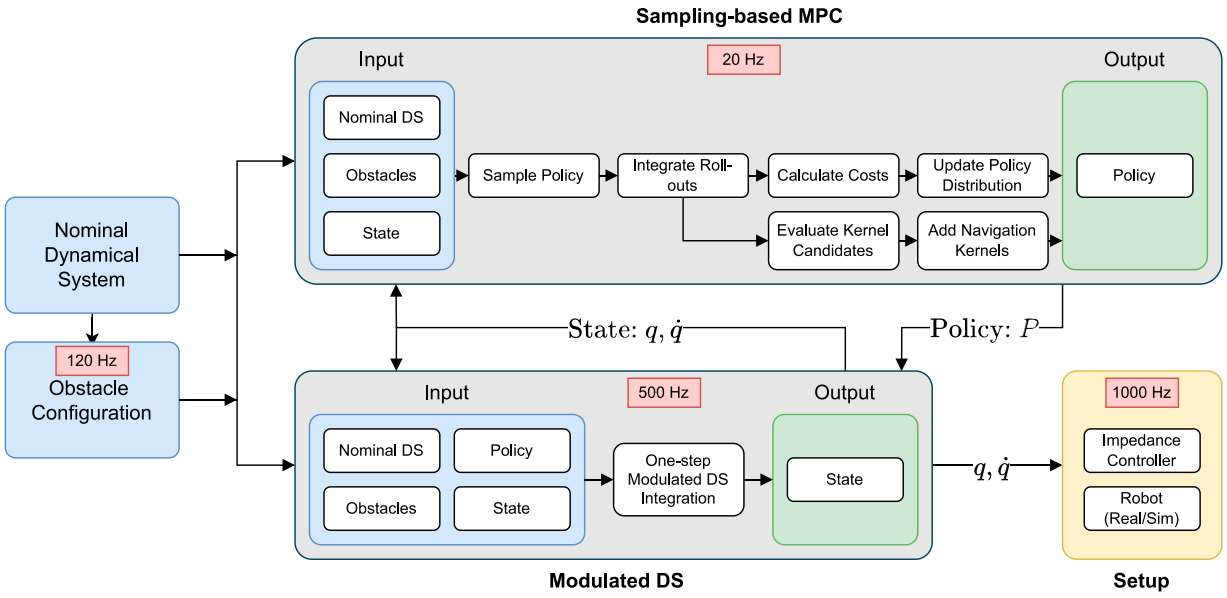


Figure 7. Flowchart of the implemented algorithm. Message passing between modules is performed asynchronously. Approximate operating frequencies for each module (shown in red boxes) are estimated for execution on Apple Silicon M2 3.7 GHz CPU.

particularly the computation of the Modulation matrix, which depends on the implicit distance function. This function is represented by a Multi-Layer Perceptron (MLP) with analytical computational complexity $O(s{\cdot}d{\cdot}n + m{\cdot}n^2)$, where s is the number of input queries; d is the input dimension (robot DoFs); m is the network depth; and n is the number of neurons per layer. Additionally, the number of inputs $s = N{\cdot}p$ can be expressed as the product of the number of robot states N and the number p of collision spheres (or points) in the robot's environment. The MPC algorithm performs these computations sequentially for H steps along the horizon, resulting in a total computational complexity for the sampling-based MPC of $O(H(s{\cdot}d{\cdot}n + m{\cdot}n^2))$. Although the integration steps in MPC can be parallelized for multiple states and obstacles, the nonlinear integration procedure requires sequential evaluation of the time dimension. Consequently, the MPC evaluation frequency does not exceed 30 Hz in our experiments.

The DS can be considered as a single-step, single-sample MPC, and its computational complexity can then be expressed as $O(d{\cdot}n + m{\cdot}n^2)$. Given the relatively small size of the MLP used ($n = 256$, $m = 4$), the DS can be evaluated at frequencies up to 500–1000 Hz.

8. Evaluation

We systematically evaluate the proposed method in a simulated environment on a 7-DoF Franka Panda robot. We aim to compare the performance of our approach with two state-of-the-art methods that are capable of reactive robot motion towards the given goal configuration while avoiding collisions with obstacles in the robot's workspace. Specifically, we compare the methods based on their ability to avoid collisions, achieve the goal and run at a high frequency. Additionally, we demonstrate the application of our method for a nonlinear joint space DS both in simulation and on a real robot.

8.1. Experimental setup

We consider a reaching task, in which the robot has to perform a reaching motion towards a goal configuration while avoiding obstacles in the workspace. We compare our method with a standard DS modulation framework introduced by Khansari-Zadeh and Billard (2012), and with a sampling-based MPC method (STORM) introduced by Bhardwaj et al. (2022). We define a single desired joint state $\boldsymbol{q}^*$ and use a simple linear dynamical system as a nominal DS:

$$\dot{\boldsymbol{q}} = -\frac{\boldsymbol{q} - \boldsymbol{q}^*}{\|\boldsymbol{q} - \boldsymbol{q}^*\|}, \tag{36}$$

that is normalized to achieve uniform velocity profile during the task execution. The normalization is not performed when the system is close to the attractor, to avoid numerical instabilities.

Algorithm 1: MPPI Modulated DS motion

Input : Nominal DS $f(\boldsymbol{q})$, attractor $\boldsymbol{q}^*$,
Cost function $c = c(\{\boldsymbol{q}, \dot{\boldsymbol{q}}\})$,
Obstacle points $\mathcal{O}$,
Initial robot state $\boldsymbol{q}_{init}$,
Output : Optimal trajectory $\{\boldsymbol{q}_s, \dot{\boldsymbol{q}}_s\}_{s=1..S}$
Modulation parameters $\{\hat{\boldsymbol{g}}_k, \hat{\boldsymbol{q}}_k\}_{k=1..K}$
Parameters: $N, H, \Delta t, \Delta\tau$

/* Initialize the iteration counter, kernel counter, and current state */
$(s, K, \boldsymbol{q}_s) \leftarrow (0, 0, \boldsymbol{q}_{init})$
/* Main loop */
while $||\boldsymbol{q}_s - \boldsymbol{q}^*|| > 0$ **do**
  /* Explore N trajectories */
  for $i \in [1, N]$ **do**
    /* Sample navigation parameters */
    for $k \in [1, K]$ **do**
      $\hat{\boldsymbol{g}}_{i,k} \leftarrow \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$
    /* Integrate for H timesteps */
    $\boldsymbol{q}_0 \leftarrow \boldsymbol{q}_s$
    for $h \in [1, H]$ **do**
      /* Evaluate nominal DS */
      $\dot{\boldsymbol{q}}_{nom} = f(\boldsymbol{q}_{h-1})$
      /* Evaluate Modulation */
      $M \leftarrow M(\boldsymbol{q}_{h-1}, \mathcal{O})$
      /* Evaluate Navigation Kernels */
      $\dot{\boldsymbol{q}}_{nav} = g(\boldsymbol{q}_{h-1}, \{\hat{\boldsymbol{g}}_{i,k}, \hat{\boldsymbol{q}}_k\}_{k=1}^{K}, \mathcal{O})$
      /* Get total velocity */
      $\dot{\boldsymbol{q}}_{i,h-1} = M(\dot{\boldsymbol{q}}_{nom} + \dot{\boldsymbol{q}}_{nav})$
      /* Integrate */
      $\boldsymbol{q}_{i,h} = \boldsymbol{q}_{i,h-1} + \dot{\boldsymbol{q}}_{i,h-1}\Delta t$
    /* Evaluate cost of trajectory */
    $c_i = c(\{\boldsymbol{q}_{i,h}, \dot{\boldsymbol{q}}_{i,h}\}_{t=1..H})$
  /* Update sampling policy */
  for $k \in [1, K]$ **do**
    $(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \leftarrow$ *Update using* c_i (Eqns. 17-19)
  /* Add new navigation kernels */
  for $i \in [1, N]$ **do**
    for $h \in [1, H]$ **do**
      if $\boldsymbol{q}_{i,h}$ *satisfies (28b)* **then**
        $K \leftarrow K + 1$
        $k^* = \min_k ||\boldsymbol{q}_{i,h} - \hat{\boldsymbol{q}}_k||_2$
        $(\hat{\boldsymbol{q}}_K, \boldsymbol{\mu}_K, \boldsymbol{\Sigma}_K) \leftarrow (\boldsymbol{q}_{i,h}, \boldsymbol{\mu}_{k^*}, \boldsymbol{\Sigma}_{k^*})$
  /* Find lowest cost */
  $i^* = \min_i c_i$
  /* Update robot state */
  $\dot{\boldsymbol{q}}_{s-1} = \dot{\boldsymbol{q}}_{i^*,0}$
  $\boldsymbol{q}_s = \boldsymbol{q}_{s-1} + \dot{\boldsymbol{q}}_{s-1}\Delta\tau$
  $s = s + 1$
return $\{\boldsymbol{q}_s, \dot{\boldsymbol{q}}_s\}_{s=1..S}, \{\hat{\boldsymbol{g}}_k, \hat{\boldsymbol{q}}_k\}_{k=1..K}$

Standard modulation and our method are designed to act in the joint space given some nominal DS, however the STORM implementation considers costs defined in task space (i.e., end-effector position and orientation). To enable a fair comparison, we redefine the goal reaching and orientation costs as a quadratic cost similar to (20). This emulates the linear DS motion in the joint space, and allows us to compare the performance of methods on the same task. We use the same cost weights and hyperparameters as provided in the publicly available implementation[4] of STORM. Additionally, we leverage our previous work (Koptev et al., 2023) that introduced a Neural Network based Γ-function for collision checking, which improves the performance (in terms of frequency) of the method.

To make meaningful comparison in the reaching scenario, we design the experiment such that final goal state $\boldsymbol{q}^*$ describes position in the left part of robot's workspace, and initial position $\boldsymbol{q}_{init}$ is located in the right. To move between these two positions robot has to traverse through the workspace in front of it, as shown in Figure 8.

For the benchmark, we consider a static cross-shaped sphere structure placed in front of the robot. This arrangement of obstacle primitives effectively segments the robot's workspace into four distinct quarters, a configuration that is representative of typical static environments or shelving scenarios encountered in robotic navigation. We investigate methods' performance for various sizes of the obstacle. Some of the sizes are shown in Figure 9. In general, as the size of the obstacle increases, the robot's workspace becomes more constrained. For each obstacle size we vary the height of the cross center and its distance from the robot across 100 reaching motions. Benchmark results are presented in Table 1. We consider reaching successful if it is performed within 10 s. Otherwise, we do not consider it in calculations of average number of iterations and trajectory time.

8.2. Discussion

The effectiveness of our proposed method in navigating in the presence of obstacles is demonstrated in Table 1, where it outperforms two other methods with consistently higher success rates. While the planning part of our algorithm is slower than STORM, the running frequency is comparable to that of a standard DS approach. It is worth noting that our algorithm runs on a CPU, while STORM efficiently leverages GPUs. However, the slow planning loop is not crucial for our method, as the primary modulated DS achieves motion reactivity at a high frequency. We should also note that this benchmark only evaluates the algorithms in quasi-static environments and does not consider their performance in dynamic scenarios. Future work could address this limitation by incorporating obstacle velocities into collision avoidance algorithms. Nonetheless, our method is capable of reacting to dynamic changes in the environment due to its high frequency of operation, as demonstrated in experiments with a real robot.

The performance parameters of the MPC component are displayed in Table 2. As mentioned earlier, the MPPI-based sampling policy optimization is only necessary when the robot trajectory comes close to possible local minima induced by obstacles. The number of trajectories where the MPC is invoked to explore exit policies increases with environmental complexity. However, in certain situations

(e.g., when the obstacle is not very close to the robot), the MPC might be unnecessary as the nominal DS delivers required collision-free reaching motion. Furthermore, for trajectories utilizing the MPC, it does not have to be employed throughout the entire trajectory but can be applied only when the robot passes by the obstacle. The corresponding MPC activation rate parameter grows with the amount of obstacles in the robot's workspace. Lastly, Table 2 reveals that an average of 2–5 navigation kernels is required to complete the reaching task in the proposed benchmark scenario. This value can be adjusted by configuring the hyperparameters such as kernel width $\widehat{\gamma}$ and kernel placement thresholds $\delta_{\{\Gamma,n,k\}}$.

In the case of a dynamic unstructured environment, it is possible that previously placed navigation kernels may become irrelevant after obstacles have moved. This issue is addressed in our method through the coefficient α_Γ (as defined in equation (31)) that deactivates the deflection as obstacle moves away. If an obstacle once again comes close to such a kernel, MPPI will automatically begin adjusting the kernel's parameters.

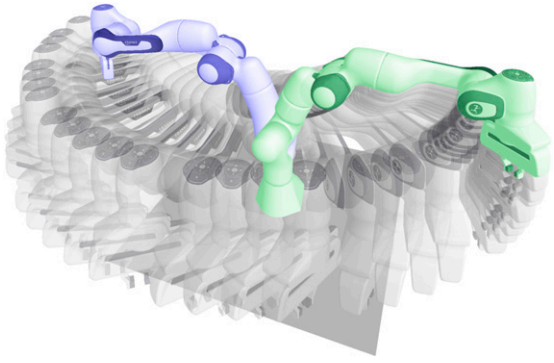


Figure 8. Reaching trajectory used in the benchmark. Robot is driven from the initial position (in blue) to the goal position (green) by linear DS defined in joint space. Concave obstacles are placed in the swept area in front of the robot to perform collision avoidance benchmark.

8.3. Real robot experiments

We demonstrate the reactive collision avoidance properties of our method on a setup involving Franka Emika. We use a simple reaching trajectory similar to the one used in simulated benchmark. We then command the robot to alternate



Figure 9. Two-, three-, and four-spheres long obstacle cross configurations used in the benchmark.

Table 1. Performance comparison of three methods for robot motion in obstructed environments with varying obstacle sizes. Values represent the average of 100 runs, with means (and standard deviations) reported. All parameters, except success rate, are averaged only for successful reaching motions.

	Success rate			Iterations			Trajectory time, s			Frequency, Hz			
Obstacle size	Ours	ModDS	STORM	Ours	ModDS	STORM	Ours	ModDS	STORM	Ours (MPC)	Ours (DS)	ModDS	STORM
2	1.00	1.00	1.00	446 (23)	451 (39)	416 (73)	0.82 (0.06)	0.74 (0.06)	4.85 (0.98)	22 (1)	548 (16)	613 (5)	86 (7)
3	1.00	1.00	0.98	476 (47)	481 (51)	452 (89)	0.96 (0.12)	0.86 (0.09)	5.34 (1.08)	22 (3)	498 (14)	562 (5)	84 (8)
4	1.00	0.86	0.82	524 (83)	534 (113)	477 (75)	0.97 (0.19)	0.86 (0.19)	6.46 (1.04)	21 (2)	546 (47)	622 (17)	74 (3)
5	1.00	0.72	0.81	563 (144)	535 (79)	467 (86)	1.18 (0.37)	0.88 (0.12)	6.05 (1.22)	22 (2)	485 (26)	607 (9)	76 (4)
6	0.99	0.65	0.80	571 (154)	554 (102)	463 (119)	1.21 (0.37)	0.98 (0.18)	5.54 (1.48)	20 (1)	473 (19)	567 (7)	75 (3)
7	0.98	0.62	0.79	626 (391)	556 (104)	454 (95)	1.36 (0.94)	0.93 (0.18)	6.26 (1.27)	19 (2)	468 (20)	598 (3)	74 (7)

Table 2. MPC activation metrics for the benchmark experiments. The columns indicate obstacle size, the number of trajectories where more than one navigation kernel was placed (i.e., MPC was required), and the corresponding proportion of iterations where the path passed by the navigation kernels along with number of kernels placed. Values are averaged over 100 runs, with means (and standard deviations) reported. MPC activation rate and number of kernels is averaged only for trajectories where MPC was active.

Obstacle size	Trajectories requiring MPC, rate	MPC activation rate	Number of kernels
2	0.11	0.42 (0.16)	1.8 (0.8)
3	0.11	0.53 (0.18)	2.7 (0.8)
4	0.28	0.64 (0.19)	2.8 (1.5)
5	0.41	0.72 (0.16)	2.6 (1.9)
6	0.51	0.72 (0.15)	2.7 (1.9)
7	0.56	0.73 (0.15)	2.7 (2.5)

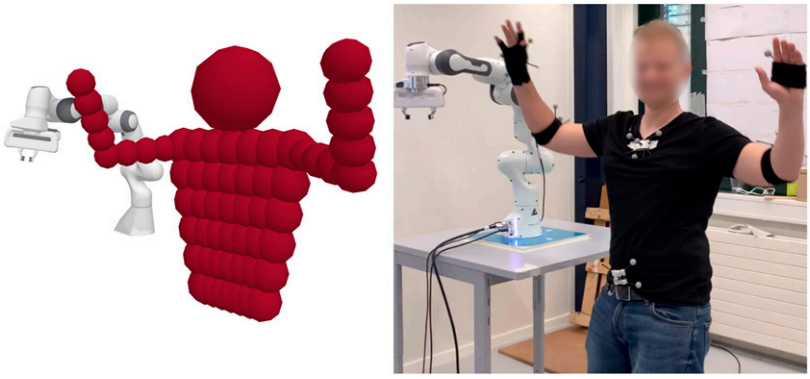


Figure 10. Spherical approximation of human upper body. 70 spheres of variable radii represent head, torso, and arms. Optitrack markers are used to determine six keypoints located on the human body.

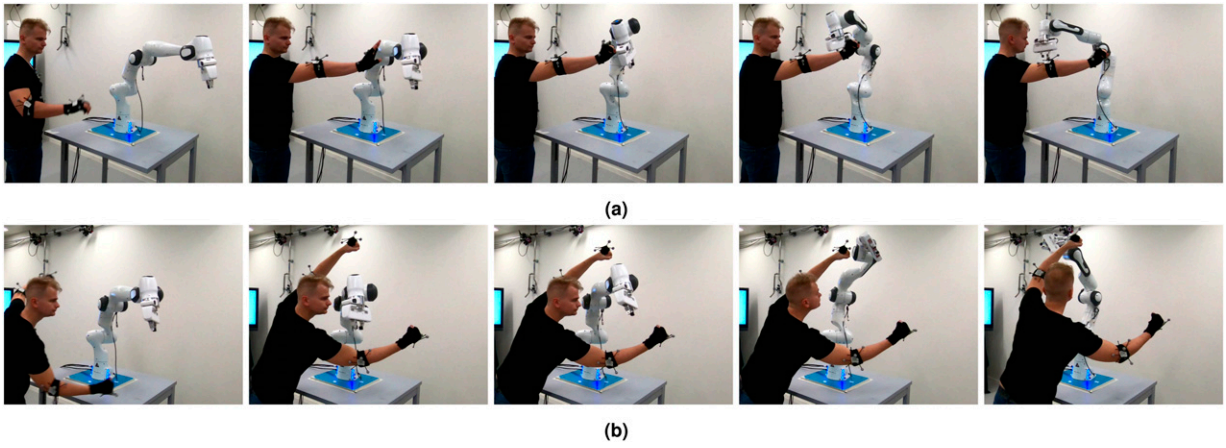


Figure 11. Two experiments in which human swiftly obstructs the robot motion with (a) raised arm, and (b) concave arm trap. Robot reactively avoids the collision and navigates the concavity, continuing the reaching task execution. Time difference between subsequent images is approximately 700 milliseconds. For more experiments please refer to the supplemental video.

between these nominal dynamics to perform cyclic motion between two attractors following the reaching pattern. We fit a human shape with 70 spheres (see Figure 10) with variable radii, and randomly obstruct the robot workspace during the task execution. The keypoints on the human are tracked with OptiTrack at a 120 Hz rate. Modulated DS with latest available policy is evaluated at frequency of at least 500 Hz, and MPPI asynchronously updates the navigation policy at approximately 20 Hz. The robot is then controlled at 1 kHz by a custom low-level torque controller. Figure 11 demonstrates the snapshots of the experiment in which human dynamically appears in the robot workspace and obstructs the motion. Multimedia attachment shows the full video of the experiment.

9. Conclusion and future work

We have presented a method to introduce deflections into a Modulated Dynamical System to enhance the navigation capabilities of the original approach proposed by Khansari-Zadeh and Billard (2012). Our method retains the properties of the Modulated DS, such as local stability and non-penetrability of obstacle boundaries.

We utilize Sampling-based Model Predictive Control to optimize the deflection parameters and leverage the Neural Signed Distance Function for real-time performance in the 7-dimensional joint space of a robotic manipulator.

Our proposed hybrid controller enables real-time, collision-free motion generation for any nominal DS motion. The underlying modulated DS can effectively control collision-free robot movement at frequencies up to 500 Hz, while the navigation policy updates at 20 Hz, allowing for path-planning in non-static and rapidly changing environments.

Future work could address some limitations of our approach, such as the generation of motion exclusively in joint-space. While this is natural for robots, it may not be ideal for real-world tasks. The hybrid controller could be enhanced by adding an Inverse Kinematics layer that calculates the desired final joint positions for each robotic task. Additionally, considering multiple DS attractors could fully exploit the redundancy of robot kinematics.

Another potential improvement involves developing a fully dynamic controller that considers not only obstacle positions but also their velocities, adding a new level of reactivity to the generated motions. This approach could be further refined by incorporating filtering and prediction of obstacle movements, enhancing the MPC policy generation.

Future work will explore advanced sampling strategies for MPPI optimization beyond standard Gaussian distributions Lambert et al. (2021); Barcelos et al. (2021). Tailoring these distributions to dynamic environments could yield quicker convergence and more nuanced motion planning, particularly in human-interactive settings. This refinement aims to bolster our method's adaptability and performance in real-world applications.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was supported by the European Research Council (ERC) (Advanced Grant agreement No. 741945, Skill Acquisition in Humans and Robots)

ORCID iDs

Mikhail Koptev https://orcid.org/0000-0001-7888-8298
Nadia Figueroa https://orcid.org/0000-0002-6873-4671

Supplemental Material

Supplemental material for this article is available online.

Notes

1. https://github.com/epfl-lasa/OptimalModulationDS.
2. $\boldsymbol{n}(\boldsymbol{q})$ is a unit vector, orthogonal to the obstacle boundary, which can be obtained, for example, by normalizing the gradient of $\Gamma(\boldsymbol{q})$.
3. https://github.com/epfl-lasa/OptimalModulationDS.
4. https://github.com/NVlabs/storm.

References

Barcelos L, Lambert A, Oliveira R, et al. (2021) Dual online stein variational inference for control and dynamics. In: *Proceedings of Robotics: Science and Systems*. UK: Virtual.

Bhardwaj M, Sundaralingam B, Mousavian A, et al. (2022) STORM: an integrated framework for fast joint-space model-predictive control for reactive manipulation. In: A Faust, D Hsu and G Neumann (eds). *Proceedings of the 5th Conference on Robot Learning, Proceedings of Machine Learning Research*. London: PMLR, Vol. 164, 750–759.

Billard A, Mirrazavi S and Figueroa N (2022) Learning for adaptive and reactive robot control: a dynamical systems approach. In: *Intelligent Robotics and Autonomous Agents Series*. Cambridge, MA: MIT Press.

Chase KJ, Ichter B, Bandari M, et al. (2021) Neural collision clearance estimator for batched motion planning. In: SM LaValle, M Lin, T Ojala, et al. (eds) *Algorithmic Foundations of Robotics XIV*. Cham: Springer International Publishing, 73–89.

Cheng CA, Mukadam M, Issac J, et al. (2018) RMPflow: a computational graph for automatic motion policy generation. In: The 13th International Workshop on the Algorithmic Foundations of Robotics, Mexico, Dec 09–11, 2018.

Domahidi A, Zgraggen AU, Zeilinger MN, et al. (2012) Efficient interior point methods for multistage problems arising in receding horizon control. In: 2012 IEEE 51st IEEE Conference on Decision and Control, Maui, HI, USA, 10–13 Dec. 2012, pp. 668–674. DOI: 10.1109/CDC.2012.6426855.

Erez T, Lowrey K, Tassa Y, et al. (2013) An integrated system for real-time model predictive control of humanoid robots. In: 2013 13th IEEE-RAS International Conference on Humanoid Robots (Humanoids), Atlanta, GA, USA, 15–17 Oct. 2013, pp. 292–299. DOI: 10.1109/HUMANOIDS.2013.7029990.

Figueroa N and Billard A (2018) A physically-consistent bayesian non-parametric mixture model for dynamical system learning. In: A Billard, A Dragan, J Peters, et al (eds) Proceedings of the 2nd Conference on Robot Learning, Proceedings of Machine Learning Research. Zurich: PMLR, Vol. 87, 927–946.

Figueroa N, Faraji S, Koptev M, et al. (2020) A dynamical system approach for adaptive grasping, navigation and co-manipulation with humanoid robots. In: 2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 2020, pp. 7676–7682. DOI: 10.1109/ICRA40945.2020.9197038.

Frasch JV, Gray A, Zanon M, et al. (2013) An auto-generated nonlinear mpc algorithm for real-time obstacle avoidance of ground vehicles. 2013 European Control Conference (ECC), Zurich, Switzerland, 17–19 July 2013, pp. 4136–4141. DOI: 10.23919/ECC.2013.6669836.

Gribovskaya E, Khansari-Zadeh S and Billard A (2011) Learning non-linear multivariate dynamics of motion in robotic manipulators. *The International Journal of Robotics Research* 30(1): 80–117.

Hansel K, Urain J, Peters J, et al. (2023) Hierarchical policy blending as inference for reactive robot control. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. Piscataway, NJ: IEEE, 10181–10188.

Hintjens P (2013) *ZeroMQ: Messaging for Many Applications*. Sebastopol, CA: O'Reilly Media, Inc.

Huber L, Billard A and Slotine JJE (2019) Avoidance of convex and concave obstacles with convergence ensured through contraction. *IEEE Robotics and Automation Letters* 4: 1462–1469.

Huber L, Slotine JJ and Billard A (2022) Avoiding dense and dynamic obstacles in enclosed spaces: application to moving in crowds. *IEEE Transactions on Robotics* 38(5): 3113–3132. DOI: 10.1109/TRO.2022.3164789.

Kavraki LE, Svestka P, Latombe JC, et al. (1996) Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation* 12(4): 566–580.

Khalil H (2002) *Nonlinear Systems*. Hoboken, NJ: Pearson Education. Prentice Hall.

Khansari-Zadeh SM and Billard A (2011) Learning stable nonlinear dynamical systems with Gaussian mixture models. *IEEE Transactions on Robotics* 27(5): 943–957.

Khansari-Zadeh SM and Billard A (2012) A dynamical system approach to realtime obstacle avoidance. *Autonomous Robots* 32(4): 433–454.

Khatib O (1986) Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research* 5(1): 90–98. DOI: 10.1177/027836498600500106.

Kingston Z, Moll M and Kavraki LE (2018) Sampling-based methods for motion planning with constraints. *Annual review of control, robotics, and autonomous systems* 1: 159–185.

Koptev M, Figueroa N and Billard A (2021) Real-time self-collision avoidance in joint space for humanoid robots. *IEEE Robotics and Automation Letters* 6(2): 1240–1247. DOI: 10.1109/LRA.2021.3057024.

Koptev M, Figueroa N and Billard A (2023) Neural joint space implicit signed distance functions for reactive robot manipulator control. *IEEE Robotics and Automation Letters* 8(2): 480–487. DOI: 10.1109/LRA.2022.3227860.

Kronander K, Khansari M and Billard A (2015) Incremental motion learning with locally modulated dynamical systems. *Robotics and Autonomous Systems* 70: 52–62.

Kuffner JJ and LaValle SM (2000) Rrt-connect: an efficient approach to single-query path planning. In: *Proceedings 2000 ICRA. IEEE International Conference on Robotics and Automation*. Piscataway, NJ: IEEE, Vol. 2, 995–1001.

Lambert A, Ramos F, Boots B, et al. (2021) Stein variational model predictive control. In: J Kober, F Ramos and C Tomlin (eds). *Proceedings of the 2020 Conference on Robot Learning, Proceedings of Machine Learning Research*. Cambridge, MA: PMLR, Vol. 155, 1278–1297.

LaValle SM (2006) *Planning Algorithms*. Cambridge: Cambridge University Press.

Li A, Cheng CA, Boots B, et al. (2019) Stable, concurrent controller composition for multi-objective robotic tasks. In: *2019 IEEE 58th Conference on Decision and Control. CDC)*. Piscataway, NJ: IEEE, 1144–1151.

Liu P, Zhang K, Tateo D, et al. (2022) Regularized deep signed distance fields for reactive motion generation. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Piscataway, NJ: IEEE, 6673–6680.

Löw T, Bandyopadhyay T, Williams J, et al. (2021) Prompt: probabilistic motion primitives based trajectory planning. In: Robotics: Science and Systems, Delft, Netherlands, Jul 15–19, 2024.

Pan J and Manocha D (2016) Fast probabilistic collision checking for sampling-based motion planning using locality-sensitive hashing. *The International Journal of Robotics Research* 35(12): 1477–1496.

Paraschos A, Daniel C, Peters JR, et al. (2013) Probabilistic movement primitives. In: *Advances in Neural Information Processing Systems*. Cambridge, MA: MIT Press.

Ratliff N, Zucker M, Bagnell JA, et al. (2009) CHOMP: gradient optimization techniques for efficient motion planning. In: *Proceedings of the 2009 IEEE International Conference on Robotics and Automation, ICRA'09*. Piscataway, NJ: IEEE Press, 4030–4035.

Ratliff ND, Issac J, Kappler D, et al. (2018) *Riemannian Motion Policies*. Ithaca, NY: arXiv.

Richter S, Jones CN and Morari M (2012) Computational complexity certification for real-time mpc with input constraints based on the fast gradient method. *IEEE Transactions on Automatic Control* 57(6): 1391–1403. DOI: 10.1109/TAC.2011.2176389.

Rimon E and Koditschek D (1992) Exact robot navigation using artificial potential functions. *IEEE Transactions on Robotics and Automation* 8(5): 501–518. DOI: 10.1109/70.163777.

Sacks J and Boots B (2022) Learning to optimize in model predictive control. In: 2022 International Conference on Robotics and Automation (ICRA), Philadelphia, PA, USA, May 23 – 27 2022, 10549–10556. DOI: 10.1109/ICRA46639.2022.9812369.

Salehian SSM, Figueroa N and Billard A (2018) A unified framework for coordinated multi-arm motion planning. *The International Journal of Robotics Research* 37(10): 1205–1232. DOI: 10.1177/0278364918765952.

Sandakalum T and Ang JMH (2022) Motion planning for mobile manipulators—a systematic review. *Machines* 10(2): 97.

Schulman J, Duan Y, Ho J, et al. (2014) Motion planning with sequential convex optimization and convex collision

checking. *The International Journal of Robotics Research* 33(9): 1251–1270.

Shavit Y, Figueroa N, Salehian SSM, et al. (2018a) Learning augmented joint-space task-oriented dynamical systems: a linear parameter varying and synergetic control approach. *IEEE Robotics and Automation Letters* 3(3): 2718–2725.

Shavit Y, Figueroa N, Salehian SSM, et al. (2018b) Learning augmented joint-space task-oriented dynamical systems: a linear parameter varying and synergetic control approach. *IEEE Robotics and Automation Letters* 3(3): 2718–2725.

Singletary A, Klingebiel K, Bourne J, et al. (2021) Comparative analysis of control barrier functions and artificial potential fields for obstacle avoidance. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Piscataway, NJ: IEEE Press, 8129–8136. DOI: 10.1109/IROS51168.2021.9636670.

Spong M, Hutchinson S and Vidyasagar M (2020) *Robot Modeling and Control*. Hoboken, NJ: Wiley.

Stasse O, Escande A, Mansard N, et al. (2008) Real-time (self)-collision avoidance task on a HRP-2 humanoid robot. In: 2008 IEEE International Conference on Robotics and Automation, Pasadena, CA, USA, 19–23 May 2008, pp. 3200–3205.

Taylor A, Singletary A, Yue Y, et al. (2020) Learning for safety-critical control with control barrier functions. *Learning for Dynamics and Control*. Berkeley, CA: PMLR, 708–717.

Urain J, Li A, Liu P, et al. (2021) Composable energy policies for reactive motion generation and reinforcement learning. *The International Journal of Robotics Research* 42(10): 827–858.

Van Wyk K, Xie M, Li A, et al. (2022) Geometric fabrics: generalizing classical mechanics to capture the physics of behavior. *IEEE Robotics and Automation Letters* 7(2): 3202–3209.

Wang Y, Figueroa N, Li S, et al. (2022) Temporal logic imitation: learning plan-satisficing motion policies from demonstrations. In: 6th Annual Conference on Robot Learning, Auckland, New Zealand, Dec 14 2022, pp. 1049–1062.

White J, Jay D, Wang T, et al. (2022) Avoiding dynamic obstacles with real-time motion planning using quadratic programming for varied locomotion modes. In: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Kyoto, Japan, 23–27 Oct. 2022. pp. 13626–13633. DOI: 10.1109/IROS47612.2022.9981268.

Williams G, Aldrich A and Theodorou EA (2017) Model predictive path integral control: from theory to parallel computation. *Journal of Guidance, Control, and Dynamics* 40(2): 344–357. DOI: 10.2514/1.G001921.

Yang Y, Pan J and Wan W (2019) Survey of optimal motion planning. *IET Cyber-Systems and Robotics* 1(1): 13–19. DOI: 10.1049/iet-csr.2018.0003.

Yoon HJ, Tao C, Kim H, et al. (2022) Sampling complexity of path integral methods for trajectory optimization. In: *2022 American Control Conference (ACC)*. Piscataway, NJ: IEEE, 3482–3487. DOI: 10.23919/ACC53348.2022.9867607.

Zhi Y, Das N and Yip MC (2022) Diffco: auto-differentiable proxy collision detection with multi-class labels for safety-aware trajectory optimization. *IEEE Transactions on Robotics* 38(5): 2668–2685.

Zucker M, Ratliff N, Dragan AD, et al. (2013) Chomp: covariant Hamiltonian optimization for motion planning. *The International Journal of Robotics Research* 32(9-10): 1164–1193.